

Energy Optimization in Open Radio Access Networks via
a
Branching Multi-Pass Parameterized Deep Q-Network
(O-RAN / BMPP-DQN Track – Draft Compile)

Gabriel Kwame Freeman

October 3, 2026

Abstract

Open Radio Access Network (O-RAN) disaggregation exposes energy-relevant control decisions – Radio Unit (RU) activation, functional split selection, transmit power, and physical resource block (PRB) allocation – that are jointly discrete and continuous and that naturally span two control timescales (Non-Real-Time and Near-Real-Time RIC). Existing deep reinforcement learning approaches to this problem handle either the discrete or the continuous side cleanly, or resolve the hybrid action space by thresholding a continuous output, which destroys the discrete decision’s own gradient signal. This thesis proposes BMPP-DQN (Branching Multi-Pass Parameterized DQN), a single coupled architecture that combines branching discrete-action decomposition with multi-pass parameterized continuous-action processing, to handle this hybrid, multi-timescale action space directly.

BMPP-DQN is formulated as a parameterized Markov Decision Process over four action branches – RU activation, split selection, transmit power, and PRB share – and evaluated in a focused, single-gNB O-RAN simulation environment against three baselines representing the discrete-only, continuous-only, and existing-parameterized-action alternatives (DQN, DDPG, and MP-DQN respectively), under a matched 3-seed, 500-episode training and evaluation protocol. The comparison is cross-checked with three independent multi-criteria decision-analysis schemes (equal-weighted and entropy-weighted TOPSIS, and VIKOR), an order-of-magnitude sensitivity sweep across the power model’s unvalidated constants, and a supplementary 10-seed statistical-power validation.

The results do not support this thesis’s original target of a $\geq 15\%$ energy saving relative to the strongest baseline: BMPP-DQN’s mean power consumption is numerically close to, not below, the best-performing baseline’s (no formal significance test was run on power specifically). What the evidence does support is a multi-criteria balance across power, QoS satisfaction, and switching stability that BMPP-DQN achieves more consistently than any single baseline, a finding that holds across every power-model perturbation tested and, substantially though not completely, across the larger 10-seed sample. A direct ablation further shows that this thesis’s two-timescale design – holding discrete RU/split decisions fixed across multiple environment steps rather than re-deciding them every step – reduces switching frequency more than sixfold without a measurable cost to reward, power, or QoS, isolating decision stability, not raw energy efficiency, as the two-timescale design’s actual contribution. These findings are reported alongside the original energy-saving target rather than in place of it, and the thesis’s remaining limitations – an unvalidated power model and reward weighting, a single-gNB scope, and a three-baseline comparison – are stated explicitly as directions for future work.

Contents

Abstract	1
1 Introduction	4
1.1 Background	4
1.2 Problem Statement	5
1.3 Research Questions	5
1.4 Research Objectives	6
1.5 Significance of the Research	6
1.6 Scope and Limitations	7
1.7 Thesis Organization	7
2 Literature Review	8
2.1 O-RAN Architecture and the Energy-Efficiency Imperative	8
2.2 Deep Reinforcement Learning for Discrete, Continuous, and Hybrid Action Spaces	9
2.3 Deep Reinforcement Learning for O-RAN Energy Optimization	10
2.4 Multi-Timescale Control in O-RAN	12
2.5 Synthesis: Research Gaps Addressed by This Thesis	12
3 System Model, Problem Formulation, and Algorithm Design	14
3.1 Network Architecture and Topology	14
3.2 Channel Model	16
3.3 Traffic Model	16
3.4 Power Consumption Model	18
3.4.1 Status of the Power Model's Numeric Constants	19
3.5 Markov Decision Process Formulation	19
3.5.1 State Space	19
3.5.2 Action Space	21
3.5.3 Transition Dynamics	21
3.5.4 Reward Function	22
3.6 Problem Formulation as Constrained Optimization	23
3.7 BMPP-DQN: Architecture	23
3.8 BMPP-DQN: Training Procedure	25

4	Simulation Results	28
4.1	Experimental Setup	28
4.2	Three Environment/Algorithm Bugs Found and Fixed	29
4.3	Training Convergence	30
4.4	Performance Comparison Against Baselines	30
4.5	Multi-Criteria Composite Analysis	34
4.6	Robustness of the Multi-Criteria Ranking	35
4.7	Heuristic and Oracle Calibration	35
4.7.1	Is the Calibration Gap a Training-Budget Artifact?	37
4.7.2	Is the Calibration Gap a Reward-Scaling Artifact?	39
4.7.3	Is the Calibration Gap an Exploration-Schedule Artifact?	39
4.8	Fair-Cadence Control: Is BMPP-DQN’s Switching Advantage Architectural?	40
4.9	Power-Model Sensitivity Analysis	41
4.10	RQ3: Effect of the Two-Timescale Design	42
4.11	Inference-Time Latency	43
4.12	Supplementary Validation at $n = 10$ Seeds	44
4.13	Discussion: Answering the Research Questions	45
5	Conclusion and Future Work	47
5.1	Summary of Contributions	47
5.2	Revisiting the Research Objectives	48
5.3	Significance of the Research, Revisited	49
5.4	Limitations	49
5.5	Future Work	50
5.6	Closing Statement	52
A	Supplementary Material	56
A.1	Default Configuration Listing	56
A.2	Per-Seed Raw Results: Canonical Comparison ($n = 3$)	60
A.3	Per-Seed Raw Results: RQ3 Two-Timescale Ablation	61
A.4	Per-Seed Raw Results: Fair-Cadence Control	62

Chapter 1

Introduction

1.1 Background

The transition from monolithic base station architectures toward Open Radio Access Networks (O-RAN) is one of the defining architectural shifts of 5G and its evolution toward 6G. O-RAN disaggregates the traditional gNB into three functional entities – the Radio Unit (RU), Distributed Unit (DU), and Central Unit (CU) – connected by open, standardized fronthaul and midhaul interfaces rather than proprietary vendor links [2]. This disaggregation, together with the introduction of RAN Intelligent Controllers (RICs), is intended to make the RAN programmable: a Non-Real-Time RIC operates on a 1–10 s control loop for slower policy and configuration decisions, while a Near-Real-Time RIC operates on a 10–100 ms loop for faster radio-resource decisions [2].

This programmability comes at the cost of substantially increased operational complexity. The RAN segment alone is estimated to account for 70–80% of a mobile network’s total energy consumption, making RAN energy efficiency a first-order operating-expenditure concern for operators and a first-order sustainability concern for the industry. Unlike a monolithic base station, an O-RAN deployment exposes energy-relevant control knobs at every disaggregation boundary: whether an RU is powered on or placed into a low-power state, which functional split is used to divide processing between the RU and the DU/CU, how much transmit power and how large a share of physical resource blocks (PRBs) each RU is allocated. These decisions are naturally structured as a *hybrid* action space: some are discrete (RU on/off, split selection) and some are continuous (transmit power, PRB share), and they occur at different natural timescales – RU activation and split selection are Non-RT-RIC-appropriate, slow decisions, while power and PRB allocation are Near-RT-RIC-appropriate, fast decisions.

Deep Reinforcement Learning (DRL) is a natural candidate for this class of sequential decision problem, since it can learn a control policy directly from interaction with a (simulated or real) environment without requiring an explicit, tractable model of the channel and traffic dynamics. However, the specific structure of the O-RAN energy-optimization problem – a hybrid, multi-timescale, discrete-continuous action space – does not fit any single standard DRL algorithm cleanly. Deep Q-Networks (DQN) and their variants handle discrete actions only; Deep Deterministic Policy Gradient (DDPG) and its successors handle continuous actions only. Parameterized-action methods such as P-DQN [3] and its refinement, Multi-Pass

DQN (MP-DQN) [4], were designed specifically to couple a discrete action choice with a continuous parameter for that action, but both process the full joint action space in a single, flat network, so one decision’s learning signal can interfere with another’s. Branching DQN (BDQ) [5] instead decomposes a discrete action space into R independent branches, reducing the size of the network’s output from $\prod_r |\mathcal{A}_r|$ to $\sum_r |\mathcal{A}_r|$, but is limited to purely discrete action spaces. No existing algorithm combines branching’s discrete-action decomposition with a parameterized method’s principled handling of coupled discrete-continuous actions.

1.2 Problem Statement

The most recent published O-RAN energy-optimization DRL framework, OREO (Open RAN Energy Optimization) [1], illustrates the consequence of this gap. OREO uses Proximal Policy Optimization (PPO) with a purely continuous action representation, recovering discrete RU on/off decisions by thresholding a continuous output – an approximation that destroys the gradient signal a genuinely discrete decision would otherwise provide, rather than a principled treatment of the hybrid action space. Its action representation is flat, giving no mechanism to prevent one decision type from interfering with another’s learning, its reward weighting is static rather than adaptive to changing network conditions, and its control loop operates at a single timescale (Non-RT RIC only), leaving the faster Near-RT RIC decisions – and any coupling between the two timescales – outside its scope.

This motivates the central problem addressed by this thesis: *how can a DRL framework combine action-space branching and multi-pass parameterized action processing to efficiently optimize energy consumption in O-RAN, while correctly handling the hybrid discrete-continuous action space that arises from RU activation, functional split selection, transmit power control, and PRB allocation under dynamic traffic conditions?* This thesis proposes and evaluates *BMPP-DQN* (Branching Multi-Pass Parameterized DQN), a DRL architecture designed specifically to close this gap.

1.3 Research Questions

This thesis is organized around three research questions:

- RQ1** How can branching action decomposition and multi-pass parameterized processing be integrated into a single DRL architecture to efficiently handle O-RAN’s hybrid discrete-continuous action space?
- RQ2** What is the energy-saving performance of BMPP-DQN relative to standard DQN, DDPG, and MP-DQN baselines in a simulated O-RAN environment?
- RQ3** How does explicitly separating slow (RU activation, split selection) and fast (power, PRB allocation) decisions across two timescales affect learning convergence and the resulting energy efficiency, relative to treating all decisions at a single timescale?

1.4 Research Objectives

The corresponding research objectives are to:

1. Design and implement the BMPP-DQN algorithm, integrating a branching discrete-action architecture with multi-pass parameterized continuous-action processing within a single coupled network.
2. Formulate O-RAN energy optimization as a parameterized Markov Decision Process (MDP) with four action branches – RU activation, functional split selection, transmit power, and PRB allocation fraction – spanning two decision timescales.
3. Implement a focused, single-gNB O-RAN simulation environment with a tractable, three-option abstraction of 3GPP functional splits.
4. Quantify BMPP-DQN’s energy-saving performance against DQN, DDPG, and MP-DQN baselines under matched traffic and evaluation conditions, against a target of $\geq 15\%$ energy savings relative to these baselines.
5. Analyze how the two-timescale, branch-separated design affects learning convergence and overall energy efficiency, relative to a single-timescale treatment of the same action space.

Objective 4’s $\geq 15\%$ target is stated here as originally planned and investigated as such; Chapter 5 reports, without qualification, that it is not met by the evidence gathered, and discusses what the gathered evidence supports instead (a multi-criteria balance across power, QoS, and switching stability, confirmed robust to a power-model sensitivity sweep) as this thesis’s actual, evidence-based contribution relative to RQ2. Objective 5 is answered directly by a single-timescale ablation reported alongside the main results (Chapter 4).

1.5 Significance of the Research

This work contributes to the DRL and O-RAN energy-optimization literatures in three ways. First, it integrates branching decomposition and multi-pass parameterized processing into a single coupled architecture – a combination that, to the best of the author’s knowledge, has not previously been proposed – extending the DRL literature on hybrid discrete-continuous action spaces. Second, it targets a specific, practically important application: O-RAN’s disaggregated RU/DU/CU architecture directly exposes the discrete-continuous, multi-timescale control problem this design is built for, and RAN energy consumption is a direct, ongoing operating-expense and sustainability concern for network operators. Third, by comparing BMPP-DQN against three representative baselines (a purely discrete method, a purely continuous method, and an existing parameterized method) under a common simulation environment, this thesis provides an incremental, evidence-based advancement over existing parameterized DRL methods, rather than an untested architectural proposal.

1.6 Scope and Limitations

The scope of this thesis is deliberately bounded to keep the research question tractable within an MPhil timeline. The study considers a single gNB, modeled with multiple RUs, one DU, and one CU, and a tractable three-level abstraction of 3GPP functional split options (approximating Options 2, 6, and 8 of 3GPP TR 38.801) rather than the full space of standardized splits. Only downlink energy optimization is considered. BMPP-DQN is compared against three baselines – DQN, DDPG, and MP-DQN – rather than the full space of prior hybrid-action methods; OREO is discussed qualitatively (Section 1.2) but not reproduced, given the complexity of its PPO-based training pipeline. All evaluation is simulation-based, using a simplified wireless channel model and a literature-informed but not independently validated power-consumption model (discussed further, with its supporting evidence, in Chapter 3). No real-world O-RAN testbed validation, multi-gNB or inter-cell coordination, or hardware-specific power measurement is attempted. These limitations are revisited in Chapter 5 in light of the simulation results presented in Chapter 4.

1.7 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 reviews the DRL and O-RAN energy-optimization literature underlying this work, expanding on the gaps identified in Section 1.2. Chapter 3 formulates the O-RAN energy-optimization problem as a parameterized MDP and presents the BMPP-DQN architecture. Chapter 4 presents the simulation setup, training procedure, and results of the comparison against the DQN, DDPG, and MP-DQN baselines. Chapter 5 summarizes the thesis’s contributions, discusses the findings relative to the research questions posed above, and outlines directions for future work.

Chapter 2

Literature Review

This chapter reviews the literature underlying the three research gaps introduced in Chapter 1: the absence of a principled treatment of O-RAN’s hybrid discrete-continuous action space (Section 2.3), the lack of an architecture that combines action-branching decomposition with multi-pass parameterized processing (Section 2.2), and the limited treatment of O-RAN’s natural multi-timescale control structure (Section 2.4). Rather than surveying each paper in isolation, the chapter is organized to build a cumulative argument: each section evaluates what a cluster of related works does and does not establish, so that Section 2.5 can state precisely which gap remains open after each cluster is accounted for.

2.1 O-RAN Architecture and the Energy-Efficiency Imperative

The O-RAN Alliance’s own architectural specification [2] formalizes the RU/DU/CU disaggregation and the two-timescale RIC structure introduced in Section 1.1, and explicitly identifies energy efficiency as a target use case for both RIC tiers. Subsequent survey work has consistently reported the RAN as the dominant contributor to mobile-network energy consumption and has catalogued AI/ML-driven control as the primary proposed remedy: Barker, Seyfi, and Afghah’s review of AI/ML in mobile edge computing and Open RAN [21] and Lassoued and Boujnah’s comprehensive review of 5G energy-efficiency strategies [22] both position learning-based RAN control as displacing static, rule-based energy-saving features (e.g., fixed sleep-mode schedules), on the grounds that traffic and channel conditions are too non-stationary for a fixed policy to remain near-optimal. Sthankiya et al.’s survey specifically of AI-driven energy optimization in next-generation RAN [7] narrows this further to O-RAN, and is useful for this thesis because it explicitly catalogues which RIC tier, which disaggregation boundary, and which learning paradigm (supervised, RL, federated) each surveyed work targets – a taxonomy this review adopts implicitly in the sections that follow. None of these three surveys, however, proposes or evaluates a specific learning architecture; they establish that the problem is real and important, and that RL is a leading candidate approach, but the specific architectural question this thesis addresses – how to represent O-RAN’s hybrid, multi-timescale action space inside an RL agent – is left open by design, as a survey’s remit.

2.2 Deep Reinforcement Learning for Discrete, Continuous, and Hybrid Action Spaces

The DRL algorithms available to address O-RAN’s control problem fall into three families, and the boundary between them is the technical crux of this thesis.

Purely discrete-action methods. Deep Q-Networks and their double/dueling refinements learn a value function over a discrete action set and select actions by maximization. In the adjacent C-RAN literature, Iqbal, Tham, and Chang [10] apply a Double DQN specifically to RRH on/off decisions, pairing it with a separate convex solver for power allocation – a two-stage design this thesis’s own discrete branches draw a Double-DQN-style target computation from (Section 1.1 already anticipates this; the mechanism is specified formally in Chapter 3). Fathy, Abood, and Hamdi [11] similarly pair a neural network (there, used for pre-processing rather than value estimation) with a convex power-allocation stage. Both works demonstrate that a discrete activation decision and a continuous power decision can be decoupled into two separately-solved stages, but neither trains the two stages jointly, end-to-end, inside a single RL agent – the coupling between them is handled procedurally (the discrete stage’s output is simply passed as an input to the convex stage), not learned. This two-stage decoupling is a reasonable engineering simplification when a convex solver is available for the continuous sub-problem, but O-RAN’s own energy-power relationship (transmit power interacting with PRB share, interference, and functional-split-dependent processing cost) is not naturally convex in the way idealized power-control problems are, which is part of the motivation for a jointly-learned approach.

Purely continuous-action methods. Deep Deterministic Policy Gradient (DDPG) and its refinements learn a deterministic policy over a continuous action space via a differentiable critic. These methods have no native mechanism for a discrete choice: a binary decision such as RU activation can only be approximated by thresholding a continuous output, which – as already noted in Section 1.2 for OREO [1] specifically – breaks the gradient signal a true discrete decision would carry, since the threshold operation’s gradient is zero almost everywhere. Shengren et al.’s benchmark of DRL algorithms for energy-systems scheduling [13] (SAC > TD3 > DDPG in training stability, evaluated on a purely continuous control problem) is useful evidence for algorithm selection *within* the continuous family, but by construction says nothing about the discrete-continuous coupling problem, since its own benchmark task has no discrete component.

Parameterized (hybrid) action-space methods. P-DQN, introduced by Xiong et al. [3], is the first architecture designed specifically for actions that are simultaneously discrete (which action to take) and continuous (a parameter of that action): a shared network produces continuous parameters for every discrete action in parallel, and a Q-network scores each (discrete action, continuous parameter) pair. This directly targets the coupling problem left unaddressed by the two clusters above. However, P-DQN’s own single-pass design has a documented flaw: the Q-network’s input includes *every* discrete action’s continuous parameters simultaneously, so the gradient used to train the parameter network for action k is contaminated by the (irrelevant) parameters of every other action $k' \neq k$ – a false cross-talk gradient rather than a genuine learning signal. Bester, James, and Konidaris’s MP-DQN refinement [4] fixes exactly this flaw with multi-pass processing: each discrete action’s Q-value

is computed from a masked input in which only that action’s own continuous parameters are non-zero, eliminating the cross-talk gradient by construction. This progression (P-DQN $\rightarrow$ MP-DQN) is the direct methodological ancestor of this thesis’s own multi-pass masking mechanism (Chapter 3).

Two applied parameterized-action papers confirm this family’s relevance to wireless resource control specifically, while also exposing its remaining limitation. Sharma and Yoon [15] apply a parameterized DQN to power allocation in massive MIMO, and Kabir, Tham, and Chang [16] apply parameterized RL to mobility-aware IoT resource allocation; a broader, unrelated IEEE study of hybrid-action RL for load balancing and energy efficiency [14] and Yan et al.’s constrained hybrid-action-space formulation [18] further confirm that parameterized RL is an active, validated methodology beyond this thesis’s specific setting. None of these four works, however, uses a *branching* decomposition of the discrete side of the action space: each treats the discrete action set as a single flat choice (e.g., “which sub-channel,” “which mobility mode”), not as a vector of R independent per-entity decisions the way RU activation naturally is (one binary decision per RU). This is precisely where P-DQN and MP-DQN’s remaining scalability limitation becomes material for O-RAN: with R RUs, S split options, and a flat parameterized representation, the discrete action set already grows combinatorially (e.g., S^R joint split choices before RU activation is even considered), reintroducing at the architectural level the same combinatorial blow-up that motivated moving from tabular Q-learning to function approximation in the first place.

Branching DQN (BDQ), proposed by Tavakoli, Pardo, and Kormushev [5], addresses exactly this combinatorial growth, but for purely discrete action spaces: by giving each of R independent decisions its own “branch” (a dueling-style Q-head sharing a common state encoder), BDQ reduces the discrete output size from $\prod_r |\mathcal{A}_r|$ to $\sum_r |\mathcal{A}_r|$ – linear rather than exponential in R . This is the second direct methodological ancestor of this thesis’s design. Critically, however, BDQ’s own formulation has no continuous-parameter mechanism at all: it is a discrete-only architecture, and its branches’ independence (each branch’s Q-value depends only on the shared state encoding, not on any other branch’s chosen action) is a design choice made for a purely discrete setting, where it is a reasonable one. Chapter 4 returns to this specific independence assumption when discussing what the empirical comparison in this thesis reveals about its consequences in an O-RAN environment where cross-RU interference genuinely couples the branches’ outcomes.

2.3 Deep Reinforcement Learning for O-RAN Energy Optimization

Having established the three architectural families in isolation, this section evaluates how each has actually been applied to O-RAN (or sufficiently O-RAN-adjacent) energy optimization specifically, and where each application still leaves the discrete-continuous, multi-branch, multi-timescale coupling problem unresolved.

OREO [1] remains the most directly comparable prior system: it targets the same problem (O-RAN energy optimization via DRL) at the same architectural ambition (a learned policy, not a heuristic), but – as detailed in Sections 1.2 and 2.2 – resolves the hybrid-action problem

only by approximation (continuous relaxation plus thresholding), uses a flat (non-branching) action representation, and operates its PPO policy at a single timescale (the Non-RT RIC’s), leaving Near-RT-RIC-scale decisions and any coupling between the two timescales outside its scope. Amiri, Wang, Shojafar, and Tafazolli [20] apply DRL to dynamic VNF (functional-split-related) splitting decisions in O-RAN specifically, which is methodologically close to this thesis’s own split-selection branch, but their formulation treats split selection as the sole learned decision and does not jointly optimize it alongside RU activation or continuous power/PRB control, so the cross-decision coupling this thesis targets is outside their scope by construction, not merely unaddressed. Bordin et al. [6] evaluate both PPO and DQN for RF-front-end energy saving in Open RAN using the **ns-0-RAN** simulator, providing useful evidence that both algorithm families are viable in a realistic O-RAN simulation environment, but their formulation, like OREO’s, does not combine branching decomposition with parameterized continuous control; it compares algorithm families on a task that itself remains discrete (RF-front-end on/off), so it does not test the hybrid-action question this thesis addresses. Rezazadeh, Chergui, Christofi, and Verikoukis’s actor-critic approach to zero-touch resource and energy control in network slicing [19] confirms actor-critic methods are viable for joint resource-energy control in a RAN-adjacent setting, but targets slice-level resource control rather than O-RAN’s specific RU/DU/CU-disaggregated action space, and does not decompose its action space by branch at all.

The closest related work to this thesis’s own proposed architecture is Lu, Yan, and Zeng’s EExApp [17], which addresses RU energy optimization in 5G O-RAN using a GNN-based RL formulation with two separate actor-critic pairs – one for RU sleep decisions, one for DU network slicing – coordinated as two interacting agents rather than one. EExApp is evidence that decomposing an O-RAN energy-control problem by decision type (sleep vs. slicing) is a productive design choice, which this thesis’s own branch decomposition shares in spirit. The architectural difference is nonetheless material: EExApp’s two decision types are each handled by an independent actor-critic pair with its own critic, whereas this thesis’s BMPP-DQN handles both decision types through a single *critic* (a non-twin **BranchingCritic**, evaluated once per branch via multi-pass masking) rather than two separately-trained critics. BMPP-DQN does still use two separate encoders internally – an upper encoder feeding the discrete branching critic and a lower encoder feeding the continuous parameter network, reflecting the two-timescale design RQ3 investigates – so the distinction from EExApp is specifically in the *value function*: one shared critic scoring both decision types’ branches, rather than two independently-trained critics each scoring one decision type. This matters for the coordination question this thesis targets: two independently trained critics must coordinate through whatever channel connects their respective agents (e.g., one agent’s action entering the other’s observed state), whereas a single shared critic scores every branch’s action against the same learned value function directly. Whether a shared-critic design in fact resolves EExApp’s own two-agent coordination problem better than two coordinated critics do is an empirical question this thesis’s own results, not this literature review, can speak to; the two designs represent genuinely different resolutions of the same underlying decomposition idea, not a strict improvement of one over the other on paper alone.

A further comparison is instructive for what it *cannot* establish. Chuang et al.’s hybrid A3C and Dueling DQN architecture [23] combines two DRL paradigms in a 5G C-RAN setting, superficially resembling this thesis’s own combination of two DRL paradigms (branching, multi-

pass parameterized processing). On inspection, however, Chuang et al. address industrial IoT task scheduling, not RU/DU/CU energy control, and their hybrid combines two purely discrete-action methods (A3C’s policy-gradient updates and Dueling DQN’s value estimation), not a discrete-continuous hybrid action space. The paper is therefore evidence only that combining two DRL paradigms within one architecture is a viable general strategy, a premise this thesis also relies on; it does not bear on the specific discrete-continuous, branch-structured coupling problem that motivates BMPP-DQN, and is engaged here only to disclose that it was considered and found not directly applicable, rather than silently omitted.

2.4 Multi-Timescale Control in O-RAN

The O-RAN Alliance’s own two-tier RIC specification [2] establishes that O-RAN’s control problem is natively multi-timescale by design, not merely as an implementation detail: the Non-RT and Near-RT RICs are specified with different control-loop periods precisely because different categories of decision (policy/configuration versus resource/scheduling) have different natural update rates. Two further works engage this structure without resolving it in the way this thesis’s RQ3 requires. Liang, Al-Tahmeesschi, Chetty, and Ahmadi’s federated-TD3 approach [8] coordinates a rApp aggregator with per-cell xApp agents – a genuine multi-tier architecture – but its tiering separates *agents by network entity* (a central aggregator versus distributed per-cell learners) rather than *decisions by timescale* within a single agent; its motivating rationale (avoiding the scalability cost of a single centralized agent as the number of cells grows) is itself cited in Section 1.6’s justification for this thesis’s own single-agent scope, but the paper does not itself study how separating decision timescales within one agent affects convergence. Sohaib et al.’s transfer-learning approach for cloud-native O-RAN [9] demonstrates that a policy trained under one traffic condition (e.g., eMBB) can be adapted to another (e.g., URLLC) with reduced retraining cost, which is a form of temporal adaptation, but again addresses adaptation *across* training episodes, not the within-episode coupling between a slow branch’s decision and a fast branch’s subsequent, dependent decisions that this thesis’s two-timescale design (RU activation and split selection held fixed for several environment steps while power and PRB allocation update every step) is built to capture. No work reviewed in this chapter evaluates, within a single learned agent, how explicitly separating slow structural decisions from fast resource decisions affects convergence relative to treating both at a single update frequency – the specific comparison this thesis’s RQ3 poses.

2.5 Synthesis: Research Gaps Addressed by This Thesis

Table 2.1 summarizes how the clusters reviewed above map onto the three gaps introduced in Section 1.2.

Three conclusions follow. First, the discrete-continuous coupling problem (Gap 1) is solved in principle by the P-DQN/MP-DQN line, but no O-RAN energy work reviewed here adopts it; O-RAN energy papers instead resolve the hybrid action space either by thresholding a continuous policy (OREO) or by decoupling discrete and continuous decisions into separately-trained components (Iqbal et al., Fathy et al., in the adjacent C-RAN literature). Second, the

Table 2.1: Mapping from reviewed literature clusters to the three research gaps this thesis addresses. A checkmark indicates the cluster resolves that dimension of the gap; a dash indicates it does not.

Cluster	Discrete-continuous coupling (Gap 1)	Branching decomposition (Gap 2)	Multi-timescale coordination (Gap 3)
DQN family [10, 11]	– (two-stage, not joint)	–	–
DDPG family [13]	– (thresholding only)	–	–
P-DQN / MP-DQN [3, 4]	✓	– (flat)	–
Branching DQN [5]	– (discrete only)	✓	–
OREO [1]	– (thresholding)	– (flat)	– (single tier)
EExApp [17]	✓ (per agent)	✓ (2 agents)	–
Federated TD3 [8]	–	–	– (agent-tiered, not decision-tiered)
BMPP-DQN (this thesis)	✓	✓	✓

branching decomposition that would let a parameterized method scale to R independent RUs (Gap 2) exists for discrete-only action spaces (BDQ) but has not been combined with multi-pass parameterized processing in any work reviewed here – EExApp’s two-agent decomposition is the closest attempt, but decomposes by decision *type* across two independently trained agents, not by *entity* within one coupled network the way BDQ decomposes by RU. Third, no work reviewed here evaluates, within a single agent, the effect of explicitly separating decision timescales on convergence (Gap 3); the closest analogues either tier by network entity (federated TD3) or by training phase (transfer learning), not by decision frequency within one training loop.

BMPP-DQN, introduced formally in Chapter 3, is positioned directly at the intersection these three gaps define: it extends BDQ’s per-RU branching with MP-DQN’s multi-pass masking so that each branch’s discrete Q-value is computed from a shared encoding without cross-talk from other branches’ continuous parameters, and it separates the resulting branches into an upper-level (RU activation, split selection) and lower-level (power, PRB allocation) network reflecting the Non-RT/Near-RT RIC timescale split the O-RAN Alliance’s own architecture specifies [2]. Chapter 4 evaluates this design against DQN, DDPG, and MP-DQN baselines and reports what the resulting comparison actually shows about each of these three design choices – including, as Chapter 5 discusses in full, a finding about branch independence that this literature review’s own treatment of BDQ’s discrete-only design in Section 2.2 anticipates as a live architectural question rather than a settled one.

Chapter 3

System Model, Problem Formulation, and Algorithm Design

This chapter formulates the O-RAN energy-optimization problem addressed by this thesis as a parameterized Markov Decision Process (MDP), specifies the channel, traffic, and power-consumption models underlying that formulation, and presents the BMPP-DQN architecture designed to solve it. Sections 3.1–3.4 describe the simulation environment (`oran.env`); Section 3.5 gives the formal MDP; Section 3.6 restates the problem as a constrained optimization and motivates a learning-based solution; Sections 3.7–3.8 present the BMPP-DQN architecture and training procedure.

3.1 Network Architecture and Topology

Consistent with the focused, single-gNB scope established in Section 1.6, the system under study consists of R Radio Units, one Distributed Unit, and one Central Unit, serving U User Equipments (UEs) in the downlink. Figure 3.1 illustrates the disaggregated architecture and the two RIC control loops that motivate this thesis’s two-timescale agent design (Section 3.7).

The default scenario (`config/oran.default.yaml`) sets $R = 4$ RUs, one DU, one CU, and $U = 8$ UEs distributed uniformly at random over a $500\text{ m} \times 500\text{ m}$ area at the start of each episode; RU positions are likewise drawn uniformly at random over the same area. This scale is a tractability-driven choice, not an independently validated “correct” size – Section 1.6 already discloses this, and Concept Note Section 10.3 situates it relative to comparable published RAN-DRL scenario scales.

Each RU can operate under one of $S = 3$ representative functional splits, abstracted as a monotonic centralization level $c \in \{0, 1, 2\}$ (loosely mapped onto 3GPP TR 38.801 Options 2, 6, and 8 respectively [24]): $c = 0$ places the most processing at the RU and requires the least fronthaul bandwidth, while $c = 2$ places the least processing at the RU (a PHY-RF split) and requires the most. This three-level abstraction is a deliberate simulation simplification, flagged *needs validation* as a design choice rather than asserted as a physically exact correspondence to the named 3GPP options – Concept Note Section 10.2 documents the supporting and conflicting evidence found for this choice in detail.

The same Gymnasium-compliant environment interface described in this chapter is used

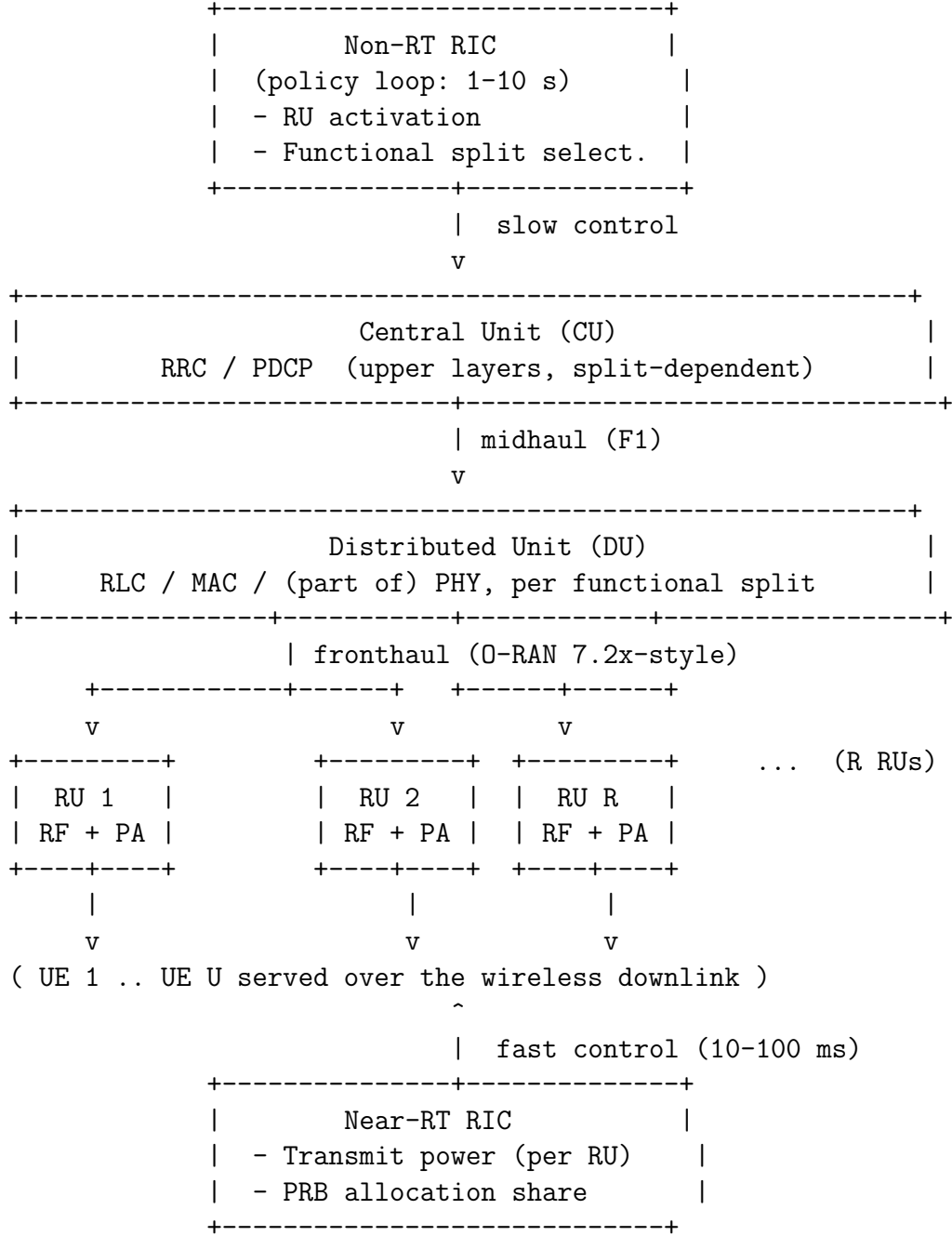


Figure 3.1: Disaggregated O-RAN architecture used in this thesis: R Radio Units (RUs) connected via fronthaul to a single Distributed Unit (DU), connected via midhaul to a single Central Unit (CU). The Non-RT RIC issues slow (1–10s) discrete decisions (RU activation, functional split); the Near-RT RIC issues fast (10–100 ms) continuous decisions (transmit power, PRB share). This timescale split motivates BMPP-DQN’s own upper/lower decomposition (Section 3.7).

both for training and evaluating BMPP-DQN and for the three baseline agents (DQN, DDPG, MP-DQN) compared in Chapter 4, ensuring that all four methods are compared under identical channel, traffic, and power-model conditions.

3.2 Channel Model

Each RU-UE link’s channel gain $h_{r,u}(t) \in \mathbb{C}$ combines log-distance path loss with independent, per-step Rayleigh small-scale fading (no shadowing term and no temporal correlation across steps – a deliberate simplification relative to the C-RAN track’s own channel model, per Concept Note Section 5.1’s “simplified SINR with path loss and Rayleigh fading”). Path loss in dB is

$$\text{PL}(d_{r,u}) = \text{PL}_0 + 10 n \log_{10} \left(\frac{d_{r,u}}{1 \text{ km}} \right), \quad (3.1)$$

where $d_{r,u}$ is the Euclidean distance between RU r and UE u (clamped to a minimum of 10 m to avoid a singular near-field value), $n = 3.5$ is the path loss exponent (`path_loss_exponent`), and

$$\text{PL}_0 = 46.3 + 33.9 \log_{10}(f_c^{\text{MHz}}) - 13.82 \log_{10}(10) \quad (3.2)$$

is a reference path loss constant at carrier frequency $f_c = 3.5 \text{ GHz}$ (`carrier_freq_ghz`), structurally in the COST-231/Hata family with a fixed effective height term. The complex channel gain is then

$$h_{r,u}(t) = 10^{-\text{PL}(d_{r,u})/20} \cdot w_{r,u}(t), \quad w_{r,u}(t) \sim \mathcal{CN}(0, 1), \quad (3.3)$$

with $w_{r,u}(t)$ a fresh, independent circularly-symmetric complex Gaussian draw at every environment step t , realized as $w_{r,u}(t) = (X + jY)/\sqrt{2}$ for independent $X, Y \sim \mathcal{N}(0, 1)$ – i.e., Rayleigh-distributed fading amplitude $|w_{r,u}(t)|$, with no correlation between successive steps. The channel bandwidth is $B = 100 \text{ MHz}$ (`bandwidth_mhz`) – raised from an original placeholder of 20 MHz after a direct empirical check found the smaller value left mean aggregate UE demand already exceeding mean achievable throughput even under a best-case, all-RUs-active reference policy, leaving no policy any real headroom to satisfy QoS regardless of learning quality; 100 MHz is a standard, precedented bandwidth for this model’s own 3.5 GHz (n78-band) carrier frequency, not an arbitrary tuning choice – and the thermal noise floor is $P_{\text{noise}} = -102 \text{ dBm}$ (`noise_power_dbm`), converted to Watts as $P_{\text{noise}}^{\text{W}} = 10^{(P_{\text{noise}} - 30)/10}$.

3.3 Traffic Model

Each UE’s downlink demand follows a deterministic diurnal envelope with a stochastic Poisson arrival count superimposed, matching the concept note’s own specification of “time-varying Poisson arrival with a daily trapezoidal pattern” (Section 5.1). The envelope $\lambda(t)$ (in packets per environment step, per UE) is a four-breakpoint trapezoid over the hour of day

$\tau = t \bmod 24$:

$$\lambda(\tau) = \begin{cases} \lambda_{\text{floor}} & \tau < t_1 \text{ or } \tau \geq t_4 \\ \lambda_{\text{floor}} + (\lambda_{\text{peak}} - \lambda_{\text{floor}}) \frac{\tau - t_1}{t_2 - t_1} & t_1 \leq \tau < t_2 \\ \lambda_{\text{peak}} & t_2 \leq \tau < t_3 \\ \lambda_{\text{peak}} - (\lambda_{\text{peak}} - \lambda_{\text{floor}}) \frac{\tau - t_3}{t_4 - t_3} & t_3 \leq \tau < t_4 \end{cases} \quad (3.4)$$

where $\lambda_{\text{floor}} = f_{\text{floor}} \lambda_{\text{peak}}$ for a floor ratio $f_{\text{floor}} = 0.2$ (`floor_ratio`), and the default breakpoints are $t_1 = 7$, $t_2 = 10$, $t_3 = 20$, $t_4 = 23$ (hours). Figure 3.2 sketches this shape.

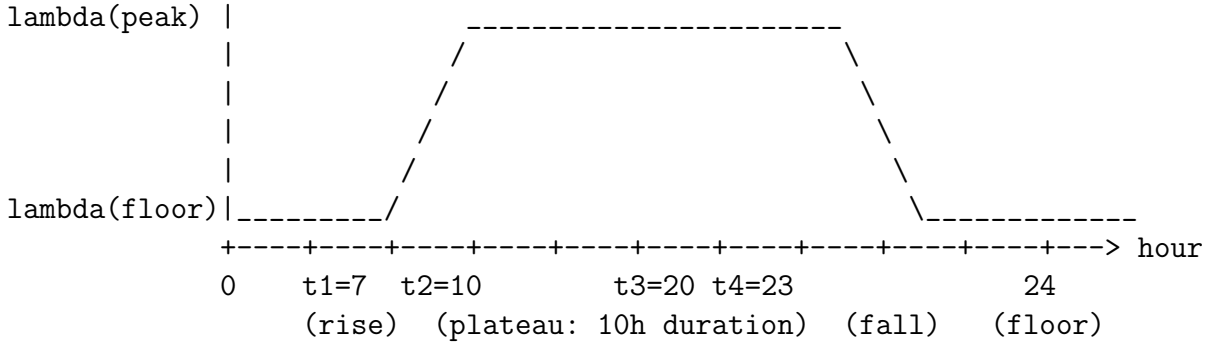


Figure 3.2: Trapezoidal diurnal traffic envelope $\lambda(\tau)$ (Eq. 3.4). The floor duration (8 h, from t_4 wrapping to t_1) and combined rise+plateau+fall duration (16 h) match ETSI TR 103 737’s own Low (8 h) and combined Medium+Busy (16 h) load-period split (Concept Note Section 10.6); the four individual breakpoints and f_{floor} itself remain unvalidated placeholders.

At each step, UE u ’s instantaneous packet arrival count is drawn $K_u(t) \sim \text{Poisson}(\lambda(\tau))$, independently across UEs, and converted to a demand rate

$$D_u(t) = K_u(t) \frac{L_{\text{pkt}}}{\Delta t} \text{ bps}, \quad (3.5)$$

where $L_{\text{pkt}} = 4 \times 10^6$ bits (`packet_size_bits`) and $\Delta t = 0.1$ s (`step_duration_s`) is the environment’s step duration, chosen to match the lower bound of the Near-RT RIC’s 10–100 ms decision rate. $\lambda_{\text{peak}} = 0.5$ and L_{pkt} are derived directly from 3GPP TR 38.864 Annex A’s FTP Model 3 (a 0.5 MB file size with a 200 ms mean inter-arrival time, i.e. 5 arrivals/s = 0.5 per 0.1 s step) [25] – a primary-source-derived value, not a placeholder guess. f_{floor} and the breakpoints t_1 – t_4 remain *needs-validation* placeholders: no source examined ties a specific diurnal timing or floor-to-peak ratio to a 5G/O-RAN small-cell scenario, though their *aggregate* floor-versus-active day-fraction split independently matches ETSI TR 103 737’s own load-period weighting (Concept Note Section 10.6).

3.4 Power Consumption Model

Total instantaneous power consumption decomposes additively across the four disaggregated components plus a switching term:

$$P_{\text{total}}(t) = P_{\text{RU}}(t) + P_{\text{DU}}(t) + P_{\text{CU}}(t) + P_{\text{FH}}(t) + P_{\text{sw}}(t). \quad (3.6)$$

RU power. For the set of active RUs $\mathcal{M}(t) = \{r : v_r(t) = 1\}$ (activation indicator $v_r(t) \in \{0, 1\}$) and inactive RUs $\mathcal{N}(t) = \{r : v_r(t) = 0\}$,

$$P_{\text{RU}}(t) = \sum_{r \in \mathcal{M}(t)} P_{\text{proc}}(c_r(t)) + \frac{1}{\eta} \sum_{r \in \mathcal{M}(t)} p_r(t) + |\mathcal{N}(t)| P_{\text{sleep}}, \quad (3.7)$$

where $c_r(t) \in \{0, 1, 2\}$ is RU r 's current split centralization level, $P_{\text{proc}}(c) \in \{10.0, 6.0, 3.0\}$ W is the active-RU processing power at level c (`p_proc_by_split_w`, decreasing as c increases, since a more centralized split shifts processing away from the RU), $\eta = 0.25$ is the power-amplifier drain efficiency (`pa_efficiency`), $p_r(t) \in [0, P_{\text{max}}]$ is RU r 's transmit power, and $P_{\text{sleep}} = 2.0$ W is the sleep-mode power of an inactive RU. $P_{\text{max}} = 33$ dBm (2 W) (`p_max_dbm`) matches the Benetel RAN550's real maximum transmit output power (total EIRP) for a Split-7.2x indoor small-cell O-RU [27] – the one constant in this power model independently confirmed by a real vendor datasheet rather than retained as an unvalidated placeholder. The active/sleep structural form itself follows the EARTH linear power model [26] – the same static-plus-load-dependent family the adjacent C-RAN power-modeling literature also uses [12] – though $P_{\text{proc}}(c)$, P_{sleep} , and η are otherwise literature-style placeholders chosen only to preserve a monotonic trade-off across split levels, not verified split-level RU wattage figures – Concept Note Section 10.5 documents nine rounds of literature verification against this specific gap, discussed further in Section 3.4.1.

DU power.

$$P_{\text{DU}}(t) = P_{\text{DU}}^{\text{static}} + \sum_{r \in \mathcal{M}(t)} P_{\text{DU,proc}}(c_r(t)), \quad (3.8)$$

with static power $P_{\text{DU}}^{\text{static}} = 50.0$ W (`p_static_w`) and per-active-RU processing power $P_{\text{DU,proc}}(c) \in \{5.0, 10.0, 20.0\}$ W (`p_per_ru_by_split_w`, *increasing* with c , since a less centralized split pushes more processing to the DU).

CU power.

$$P_{\text{CU}}(t) = P_{\text{CU}}^{\text{static}} + P_{\text{CU}}^{\text{dyn}} |\mathcal{M}(t)|, \quad (3.9)$$

with $P_{\text{CU}}^{\text{static}} = 30.0$ W (`p_static_w`) and $P_{\text{CU}}^{\text{dyn}} = 1.0$ W per active RU (`p_dyn_per_ru_w`).

Fronthaul power.

$$P_{\text{FH}}(t) = \begin{cases} 0 & \mathcal{M}(t) = \emptyset \\ P_{\text{FH}}^{\text{common}} + \sum_{r \in \mathcal{M}(t)} P_{\text{FH,proc}}(c_r(t)) & \text{otherwise,} \end{cases} \quad (3.10)$$

with $P_{\text{FH}}^{\text{common}} = 10.0$ W (`p_common_w`) and per-active-RU fronthaul power $P_{\text{FH,proc}}(c) \in \{3.0, 6.0, 15.0\}$ W (`p_per_ru_by_split_w`, increasing with c , reflecting a less-centralized split's larger fronthaul bandwidth requirement).

Switching cost. Let $\mathbb{K}[\cdot]$ denote the indicator function. Between consecutive steps,

$$P_{\text{sw}}(t) = P_{\text{sw}}^{\text{RU}} \sum_{r=1}^R \mathbb{K}[v_r(t) \neq v_r(t-1)] + P_{\text{sw}}^{\text{split}} \sum_{r=1}^R \mathbb{K}[c_r(t) \neq c_r(t-1)] \cdot v_r(t) v_r(t-1), \quad (3.11)$$

with $P_{\text{sw}}^{\text{RU}} = 2.0 \text{ W}$ (`p_switch_w`) per activation-state flip and $P_{\text{sw}}^{\text{split}} = 1.0 \text{ W}$ (`p_switch_split_w`) per split change – the latter counted only for RUs active in *both* the previous and current step, since an RU that just switched on or off has not meaningfully “changed” a split choice in a way that costs anything beyond the activation flip already counted.

3.4.1 Status of the Power Model’s Numeric Constants

Every numeric constant in Equations 3.7–3.11 except P_{max} is a literature-style placeholder retained specifically to preserve a monotonic energy trade-off across split levels $c \in \{0, 1, 2\}$ – not a verified physical measurement. This is disclosed here, rather than deferred silently to a limitations section, because Quality Gate 3 (Reproducibility) and Gate 6 (Code-Text Consistency) both require this chapter to state plainly what is fact versus placeholder. Concept Note Section 10.5 documents nine independent literature-verification passes against this specific gap (3GPP TR 38.864’s static-plus-dynamic model family, a real small-cell O-RU datasheet, component-decomposed O-RAN testbed measurements, and analytical O-DU/O-CU power models, among others); the consistent finding across all nine is that no source combines a varied 3GPP split option with matching, component-decomposed (RU vs. DU vs. CU vs. fronthaul) absolute power in Watts at this model’s small-cell scale – the specific combination needed to validate Equations 3.7–3.10’s per-split arrays directly. Real measured hardware power (macro-cell O-RUs, enterprise-class O-DU/O-CU compute hosts, and a real commercial multi-band O-RAN test line) runs 10–50 $\times$ above this model’s own small-cell placeholder scale, and this gap is not closed by rescaling, since no source specifies what absolute scale a $R = 4$ -RU simulation should use. Chapter 4 reports a sensitivity analysis perturbing these constants within the ranges the literature review brackets, to test whether the comparative conclusions in Chapter 4 depend on any single unvalidated constant.

3.5 Markov Decision Process Formulation

The O-RAN energy-optimization problem is formulated as a parameterized MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ with a hybrid discrete-continuous action space, following the four action branches specified in Concept Note Section 10.1. Figure 3.3 sketches the agent-environment interaction loop this section formalizes.

3.5.1 State Space

The state $s(t) \in \mathbb{R}^d$, $d = R \cdot U + R + R \cdot S + U + 4$, concatenates:

$$s(t) = \left[|h_{r,u}(t)|_{r,u}, v(t-1), \text{onehot}(c(t-1)), D(t)/10^6, \frac{P_{\text{total}}(t-1)}{1000}, \frac{\tau(t)}{24}, \bar{T}, \frac{\bar{P}}{1000} \right], \quad (3.12)$$

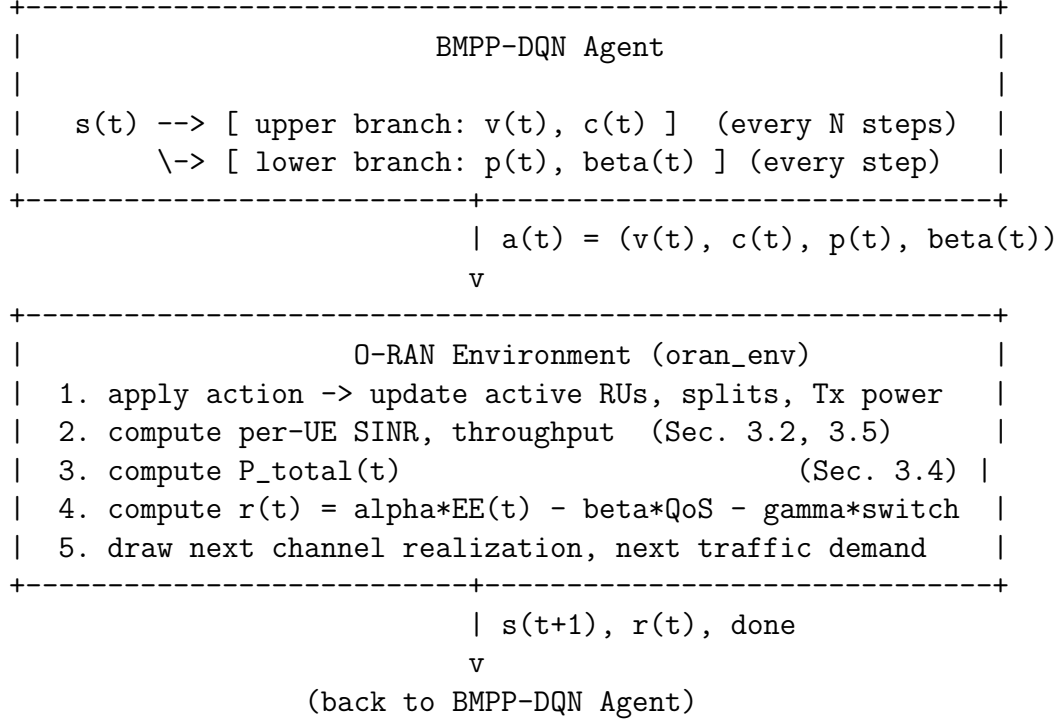


Figure 3.3: Agent-environment interaction loop. The agent’s discrete branches (RU activation v , split c) are recomputed only every $N = \text{upper_level_period_steps}$ environment steps, while its continuous branches (power p , PRB share β) are recomputed every step – realizing the two-timescale design motivated in Section 3.1. The environment itself is timescale-agnostic: `step()` always accepts all four action components every call.

where $|h_{r,u}(t)|_{r,u}$ flattens the $R \times U$ channel-gain magnitude matrix, $v(t-1) \in \{0,1\}^R$ is the previous step's RU activation vector, $\text{onehot}(c(t-1)) \in \{0,1\}^{R \times S}$ one-hot encodes each RU's previous split choice, $D(t)/10^6 \in \mathbb{R}^U$ is the current step's UE demand vector in Mbps, $P_{\text{total}}(t-1)/1000$ is the previous step's total power in kW, $\tau(t)/24$ is the hour of day normalized to $[0, 1]$, and $\bar{T}$, $\bar{P}/1000$ are the rolling-window mean throughput (Mbps) and mean power (kW) over the last $\min(t, N)$ steps – the mechanism by which lower-level (fast) performance metrics are propagated into the state the upper-level (slow) decision observes, per Concept Note Section 5.2. At the default scenario scale ($R = 4$, $U = 8$, $S = 3$), $d = 32 + 4 + 12 + 8 + 4 = 60$.

3.5.2 Action Space

The action $a(t) = (v(t), c(t), p(t), \beta(t))$ has four branches:

$$v(t) \in \{0, 1\}^R \quad \text{RU activation (discrete, upper timescale)} \quad (3.13)$$

$$c(t) \in \{0, 1, \dots, S-1\}^R \quad \text{Functional split choice (discrete, upper timescale)} \quad (3.14)$$

$$p(t) \in [0, P_{\max}]^R \quad \text{Transmit power per RU (continuous, lower timescale)} \quad (3.15)$$

$$\beta(t) \in [0, 1]^R, \sum_r \beta_r(t) = 1 \quad \text{PRB share among active RUs (continuous, lower timescale)} \quad (3.16)$$

Inactive RUs' power and split entries are forced to zero by the environment ($p_r(t) \leftarrow 0$, $c_r(t) \leftarrow 0$ for $r \notin \mathcal{M}(t)$) before the reward and power computations of Section 3.4 are evaluated, and the raw PRB vector is re-normalized over active RUs only so that Eq. 3.16's constraint holds with respect to $\mathcal{M}(t)$, not the full RU set.

3.5.3 Transition Dynamics

The channel realization is redrawn independently every step (Eq. 3.3; no temporal correlation), and the UE demand vector is redrawn from the traffic envelope at the new hour (Eq. 3.5):

$$P(s(t+1) \mid s(t), a(t)) = P(H(t+1)) P(D(t+1) \mid \tau(t+1)), \quad (3.17)$$

i.e., the next channel and demand realizations are conditionally independent of the current state given the (implicit) hour-of-day clock, which itself advances deterministically by this step's real duration in hours, $\tau(t+1) = (\tau(t) + \Delta t_{\text{step}}/3600) \bmod 24$, with $\Delta t_{\text{step}} = 0.1$ s (`step_duration_s`, the same constant the Poisson arrival process in Eq. 3.5 is scaled by). A 100-step episode therefore drifts the clock by only ≈ 10 s in total: within one episode the diurnal regime is effectively fixed at whichever hour `reset()` drew, and the full diurnal range is seen across episodes/seeds via that random draw, not within a single short episode. An earlier implementation advanced the clock by a full simulated hour per step regardless of Δt_{step} – compressing an entire day into ≈ 24 steps and decoupling the two-timescale design's decision cadence (Section 3.8) from any real correspondence to Δt_{step} or to the Non-RT/Near-RT RIC timescales motivating it (Section 1.1); this was identified and fixed during the investigation reported in Chapter 4.

3.5.4 Reward Function

Each UE u is served by its strongest active RU, $r^*(u) = \arg \max_{r \in \mathcal{M}(t)} p_r(t) |h_{r,u}(t)|^2$, and experiences co-channel interference from every other simultaneously-active RU:

$$\text{SINR}_u(t) = \frac{p_{r^*(u)}(t) |h_{r^*(u),u}(t)|^2}{\left(\sum_{r \in \mathcal{M}(t)} p_r(t) |h_{r,u}(t)|^2 \right) - p_{r^*(u)}(t) |h_{r^*(u),u}(t)|^2 + P_{\text{noise}}^W}, \quad (3.18)$$

$$C_u(t) = \frac{\beta_{r^*(u)}(t)}{K_{r^*(u)}(t)} B \log_2(1 + \text{SINR}_u(t)), \quad (3.19)$$

where $C_u(t)$ is UE u 's achievable Shannon capacity given the PRB share $\beta_{r^*(u)}(t)$ its serving RU has allocated it, divided among the $K_{r^*(u)}(t) \geq 1$ UEs that RU is *simultaneously* serving this step (an equal per-UE split of that RU's own allocated bandwidth). An earlier implementation omitted the $K_{r^*(u)}(t)$ divisor, so every UE a multi-UE RU served was scored as if it alone held that RU's entire allocated share – summed capacity scaled with how many UEs an RU served rather than being bounded by the RU's own bandwidth allocation, inflating $C_{\text{total}}(t)$ (and therefore $\text{EE}(t)$ and the QoS terms below) whenever $K_r(t) > 1$, which happens routinely at this scenario's $U/R = 2$ average UE-per-RU ratio; this was identified and fixed during the investigation reported in Chapter 4. Total throughput is $C_{\text{total}}(t) = \sum_u C_u(t)$ (Mbps, after unit conversion), and energy efficiency is

$$\text{EE}(t) = \frac{C_{\text{total}}(t)}{P_{\text{total}}(t) + \epsilon} \text{ [Mbit/Joule]}, \quad \epsilon = 10^{-6}, \quad (3.20)$$

with ϵ a numerical-stability term, not a design parameter. The reward is

$$r(t) = \alpha \text{EE}(t) - \beta_{\text{qos}} \sum_{u=1}^U \frac{\max(0, D_u(t) - C_u(t))}{10^6} - \gamma_{\text{sw}} n_{\text{sw}}(t), \quad (3.21)$$

where $\alpha = 1.0$ (**alpha_energy**), $\beta_{\text{qos}} = 10.0$ (**beta_qos**) weights the total QoS shortfall in Mbps, and $\gamma_{\text{sw}} = 0.5$ (**gamma_switch**) weights the exact count of switching events $n_{\text{sw}}(t)$ – RU activation flips plus split changes among RUs active in both the previous and current step (the same event count that determines $P_{\text{sw}}(t)$ in Eq. 3.11, so a switching event is penalized once in the reward via γ_{sw} and, separately, through its contribution to $P_{\text{total}}(t)$ inside $\text{EE}(t)$ itself). Unlike every power/traffic/split constant in Sections 3.2–3.4, α , β_{qos} , and γ_{sw} have not been subjected to a literature check or sensitivity analysis, despite directly defining the optimization objective every result in Chapter 4 is measured against; they are chosen only to preserve a reasonable relative magnitude (β_{qos} large enough that a QoS violation dominates the smaller throughput-per-watt term) and remain flagged *needs validation*, unlike P_{max} and η in Section 3.4. Table 3.1 summarizes the state, action, and reward quantities defined above.

Table 3.1: Summary of the MDP formulation’s state, action, and reward components, with default values at $R = 4$, $U = 8$, $S = 3$.

Component	Symbol	Dimension / Default
State	$s(t)$	$\mathbb{R}^{60}$ (Eq. 3.12)
Action – RU activation	$v(t)$	$\{0, 1\}^4$
Action – split choice	$c(t)$	$\{0, 1, 2\}^4$
Action – transmit power	$p(t)$	$[0, 2\text{ W}]^4$
Action – PRB share	$\beta(t)$	$[0, 1]^4$, sums to 1 over active RUs
Reward weights	$\alpha, \beta_{\text{qos}}, \gamma_{\text{sw}}$	1.0, 10.0, 0.5
Discount factor	γ	0.99

3.6 Problem Formulation as Constrained Optimization

The underlying control problem this MDP approximates is

$$\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t r(t) \right] \quad (3.22)$$

$$\text{s.t. } 0 \leq p_r(t) \leq P_{\max}, \quad \forall r, t \quad (3.23)$$

$$v_r(t) \in \{0, 1\}, \quad \forall r, t \quad (3.24)$$

$$c_r(t) \in \{0, \dots, S-1\}, \quad \forall r, t \quad (3.25)$$

$$\sum_{r \in \mathcal{M}(t)} \beta_r(t) = 1, \quad \forall t. \quad (3.26)$$

The QoS constraint $C_u(t) \geq D_u(t)$ is not enforced as a hard constraint but folded into the objective as the shortfall penalty in Eq. 3.21, since a hard per-step QoS constraint is generally infeasible under bursty Poisson demand and would otherwise force every RU permanently active. A model-free RL approach is preferred over a convex or heuristic solver for this problem for the same reasons argued in the adjacent C-RAN literature (Chapter 2): the channel (Eq. 3.3) and traffic (Eq. 3.4–3.5) processes are stochastic and non-stationary over the diurnal cycle, the optimal policy is plausibly history-dependent (via the propagated rolling-window state terms in Eq. 3.12), and Eq. 3.22 is not convex in (v, c, p, β) jointly, since $\text{SINR}_u(t)$ (Eq. 3.18) is a ratio of terms each RU’s own decision variables enter both the numerator (its own signal) and every other UE’s denominator (as interference).

3.7 BMPP-DQN: Architecture

BMPP-DQN addresses Eq. 3.22–3.26 by combining Branching DQN’s per-RU discrete decomposition (Chapter 2, Section 2.2) with MP-DQN’s multi-pass masking, split across two independently-parameterized encoders that realize the upper/lower timescale separation of Figure 3.1. Figure 3.4 shows the resulting network architecture.

Encoders. $h_{\text{up}}(s | \theta_{\text{up}})$ and $h_{\text{low}}(s | \theta_{\text{low}})$ are separate two-layer feedforward networks (hidden sizes [128, 64], ReLU activation, LayerNorm after each layer), with independent weights – *not* a single shared encoder feeding both branches. This is a deliberate departure

from a literal reading of Concept Note Section 5.2 (which describes “two Q-networks,” one per timescale); Section 10.4’s implementation addendum records the correction actually realized in code: one *critic* (shared across both discrete branch-groups), fed by the upper encoder, and one continuous parameter network, fed by the lower encoder.

Continuous parameter network. From the lower encoder’s feature $\text{feat}_{\text{low}} \in \mathbb{R}^{64}$, a linear power head followed by a sigmoid produces a per-RU power ratio $\hat{p}(t) \in [0, 1]^R$ (scaled to $p(t) = \hat{p}(t) \cdot P_{\text{max}}$ before being sent to the environment), and a linear PRB head followed by a softmax produces $\hat{\beta}(t) \in [0, 1]^R$ with $\sum_r \hat{\beta}_r(t) = 1$ (over *all* R RUs; the environment re-normalizes this over active RUs only, as noted after Eq. 3.16).

Branching critic. For each RU r , a masked pass encodes only that RU’s own $(\hat{p}_r(t), \hat{\beta}_r(t))$ pair through a small linear layer (`param_encoder`, output dimension 64), concatenates it with the upper encoder’s feature, and passes the result through a fusion layer to produce $\text{fused}_r \in \mathbb{R}^{64}$. Two dueling head groups then read fused_r :

$$Q_r^v(s, k) = V^v(\text{fused}_r) + \left(A_r^v(\text{fused}_r)_k - \frac{1}{2} \sum_{k'} A_r^v(\text{fused}_r)_{k'} \right), \quad k \in \{0, 1\}, \quad (3.27)$$

and analogously $Q_r^c(s, k)$ for $k \in \{0, \dots, S-1\}$, where the value head $V(\cdot)$ is shared across all R branches within a branch-group (activation or split) while each branch has its own advantage head $A_r(\cdot)$ – the standard Branching DQN factorization (Chapter 2), reproduced here per branch-group rather than per single discrete action set. Only branch r ’s own output slice is kept from pass r ; the R passes are otherwise independent forward computations through the same critic weights.

3.8 BMPP-DQN: Training Procedure

Figure 3.5 illustrates the two-timescale decision and update cadence; Algorithm 1 gives the full training loop.

env step:	1	2	3	4	5	...	N	N+1	N+2	...			
discrete (v,c):	D	--	held	--	held	--	held	--	D	--	held	--	...
continuous(p,b):	D	D	D	D	D	...	D	D	D	...			
update_lower():	U	U	U	U	U	...	U	U	U	...			
update_upper():	U	U	U	U	U	...	U	U	U	...	(*)		
upper buffer push:							*				(window-sum reward)		
lower buffer push:	*	*	*	*	*	...	*	*	*		(per-step reward)		

Figure 3.5: Two-timescale decision and update cadence (`upper_level_period_steps`= N = 10 by default). “D” marks a freshly-made decision; “held” marks a cached decision replayed unchanged. Both `update_lower()` and `update_upper()` are in fact called *every* environment step by the training loop (`oran_training/train_bmpp_dqn.py`) – the two-timescale design is realized entirely through the decision/action cadence above and through the upper replay buffer’s refill rate (marked (*): a new upper-buffer transition, with reward summed over the N intervening steps, is pushed only once per window), *not* through a differential gradient-update frequency. Concretely, this means `update_upper()` resamples and retrains on the same, slowly growing upper buffer up to N times before that buffer receives a single new transition.

Two design choices in the training procedure are worth stating explicitly, since both were adopted to fix genuine training-instability failure modes documented during development (Chapter 4 reports the before/after evidence):

1. **Reward normalization.** Both replay buffers store a reward divided by a running (Welford’s-algorithm) estimate of that buffer’s own reward stream standard deviation, *never* mean-shifted (only scaling, not shifting, preserves which policy is reward-optimal). The upper buffer’s window-summed reward is normalized against its own, separately-tracked running statistic, since it is naturally larger in magnitude than the lower buffer’s per-step reward.
2. **Target-network bootstrapping for the continuous actor.** The continuous parameter network’s update maximizes the branching critic’s greedy activation-branch value, but bootstraps this maximization off the *frozen*, Polyak-averaged `critic_target` / `upper_encoder_target` rather than the online critic being updated every step from this same actor’s own output – avoiding a tight online-online feedback loop between the two networks.

Default hyperparameters (`config/oran.default.yaml`): discount $\gamma = 0.99$, Polyak rate $\tau = 0.005$, decision period $N = 10$, batch size 128, learning rates 10^{-4} (critic) and 3×10^{-4} (continuous actor), ϵ decayed from 1.0 to 0.01 at rate 0.995 per episode, continuous exploration noise std decayed from 0.1 to 0.01 at the same rate, and gradient norm clipping at 1.0. Explicitly *not* present, per the concept note’s own scope (Section 10.4): twin critics, target-policy-smoothing noise, and delayed (policy-lag-gated) actor updates – the TD3-style machinery the C-RAN track’s own Branching MP-DQN architecture adds and this track’s concept note deliberately omits. Chapter 4 reports what this design, evaluated against the DQN, DDPG, and MP-DQN baselines under the model specified in this chapter, actually achieves.

Algorithm 1 BMPP-DQN training loop

```

1: Input: upper encoder  $\theta_{\text{up}}$ , lower encoder  $\theta_{\text{low}}$ , branching critic  $\theta_Q$ , continuous parameter net  $\phi$ ; target
   copies of all four; replay buffers  $\mathcal{D}_{\text{low}}$ ,  $\mathcal{D}_{\text{up}}$ ; decision period  $N$ ; Polyak rate  $\tau$ 
2: for episode = 1 to  $E$  do
3:    $s \leftarrow \text{env.reset}()$ ; steps_since_decision  $\leftarrow 0$ 
4:   for  $t = 0$  to  $T - 1$  do
5:      $\text{feat}_{\text{low}} \leftarrow h_{\text{low}}(s | \theta_{\text{low}})$ ;  $(\hat{p}, \hat{\beta}) \leftarrow \text{CONTINUOUSPARAMNET}(\text{feat}_{\text{low}} | \phi)$ 
6:     add exploration noise to  $\hat{p}$  (Gaussian, std decayed once per episode)
7:     if steps_since_decision = 0 then
8:        $\text{feat}_{\text{up}} \leftarrow h_{\text{up}}(s | \theta_{\text{up}})$ ;  $(v^{\text{greedy}}, c^{\text{greedy}}) \leftarrow \arg \max_k Q_r^{v,c}(\text{feat}_{\text{up}}, x)$  for each  $r$   $\triangleright$  multi-pass,
Eq. 3.27
9:       for  $r = 1$  to  $R$  do  $\triangleright$  independent per-branch  $\epsilon$ -greedy (Tavakoli et al. 2018)
10:        with prob.  $\epsilon$ :  $v_r \leftarrow \text{uniform}\{0, 1\}$ ; otherwise:  $v_r \leftarrow v_r^{\text{greedy}}$ 
11:        with prob.  $\epsilon$ :  $c_r \leftarrow \text{uniform}\{0, \dots, S - 1\}$ ; otherwise:  $c_r \leftarrow c_r^{\text{greedy}}$ 
12:      end for
 $\triangleright$  each of the  $2R$  branches rolls its own coin – a single shared coin for all branches jointly (explore
all-or-nothing) was an early implementation defect, fixed and regression-tested; see Chapter 4
13:      cache  $(v, c)$ 
14:    else
15:      replay cached  $(v, c)$ 
16:    end if
17:     $a \leftarrow (v, c, \hat{p} \cdot P_{\text{max}}, \hat{\beta})$ ;  $s', r, \text{done} \leftarrow \text{env.step}(a)$ 
18:    push  $(s, (\hat{p}, \hat{\beta}), r / \hat{\sigma}_{\text{low}}, s', \text{done})$  to  $\mathcal{D}_{\text{low}}$   $\triangleright$  normalized
19:    accumulate  $r$  into the pending upper-window sum
20:    if steps_since_decision =  $N - 1$  or done then
21:      push  $(s_{\text{decision}}, v, c, \sum r / \hat{\sigma}_{\text{up}}, s', \text{done})$  to  $\mathcal{D}_{\text{up}}$ 
22:    end if
23:    UPDATELOWER  $\triangleright$  every step
24:    UPDATEUPPER  $\triangleright$  also every step – resamples the same, slowly-growing upper buffer up to  $N$ 
times per new transition; the two-timescale separation is realized only through the decision cadence and
buffer refill rate above, not through gradient-update frequency
25:    steps_since_decision  $\leftarrow (\text{steps\_since\_decision} + 1) \bmod N$ ;  $s \leftarrow s'$ 
26:  end for
27:  decay  $\epsilon$  and the continuous exploration noise std
28: end for

29: function UPDATELOWER
30:   sample batch from  $\mathcal{D}_{\text{low}}$ ; recompute  $(\hat{p}, \hat{\beta})$  at sampled states via  $\phi, \theta_{\text{low}}$ 
31:    $Q^v \leftarrow$  multi-pass critic evaluated via target nets  $\theta_{\text{up}}^-, \theta_Q^-$  at these  $(\hat{p}, \hat{\beta})$ 
32:   param_loss  $\leftarrow -\mathbb{E}[\max_k Q_r^v(k)]$ ; backprop through  $\phi, \theta_{\text{low}}$  only
33:   soft-update  $\theta_{\text{low}}^- \leftarrow \tau \theta_{\text{low}} + (1 - \tau) \theta_{\text{low}}^-$ ;  $\phi^- \leftarrow \tau \phi + (1 - \tau) \phi^-$ 
34: end function

35: function UPDATEUPPER
36:   sample batch from  $\mathcal{D}_{\text{up}}$ 
37:    $(v^*, c^*) \leftarrow \arg \max$  of online multi-pass critic at  $s'$   $\triangleright$  Double-DQN: online selects
38:    $y \leftarrow r + \gamma(1 - \text{done}) Q^-(s', v^*, c^*)$   $\triangleright$  target net evaluates
39:   critic_loss  $\leftarrow \text{MSE}(Q_{\theta}(s, v, c), y)$ , summed over both branch-groups
40:   backprop through  $\theta_{\text{up}}, \theta_Q$ ; soft-update  $\theta_{\text{up}}^-, \theta_Q^-$ 
41: end function

```

Chapter 4

Simulation Results

4.1 Experimental Setup

All results in this chapter use the environment, power model, and BMPP-DQN architecture specified in Chapter 3 (including the three corrections disclosed in Section 4.2 below), instantiated at this track’s single focused scenario: $R = 4$ RUs, $U = 8$ UEs, one DU, one CU, and $S = 3$ functional-split options (`config/oran_default.yaml`). BMPP-DQN is compared against three baselines – a purely discrete method (DQN, thresholding the continuous actions), a purely continuous method (DDPG, relaxing the discrete actions), and an existing parameterized-action method (MP-DQN, processing the full joint discrete-continuous action space in a single flat network, unlike BMPP-DQN’s per-branch decomposition) – under matched training and evaluation conditions: 500 training episodes of 100 environment steps each, identical traffic and channel realizations per seed, and the hyperparameters listed in Section 3.8’s Default Hyperparameters paragraph. Training used GPU-accelerated PyTorch on a RunPod-hosted A40 instance.

The canonical protocol evaluates every method over $n = 3$ random seeds (42, 123, 456), per Concept Note Section 5.3 and this thesis’s own Quality Gate 5 as originally scoped. $n = 3$ is a thin basis for statistical power; Section 4.12 reports what a supplementary $n = 10$ -seed validation found under the *pre-fix* environment, with an explicit caveat that it has not been rerun against the correction below.

Five operational metrics are reported for every method: mean episodic reward (Eq. 3.21), mean total power P_{total} in watts, two distinct QoS satisfaction rates, switching frequency $n_{\text{sw}}(t)$ averaged per step, and aggregate downlink throughput in Mbps. The two QoS rates require separate definitions because they differ sharply in practice: *QoS Rate* (“QoS Strict” in the multi-criteria tables) is the fraction of evaluation steps at which *every* UE is simultaneously satisfied ($\max_u \max(0, D_u(t) - C_u(t)) = 0$ for all u); *Per-UE QoS* is the mean, per step, of the *fraction* of UEs that are individually satisfied. Because the strict criterion requires $U = 8$ roughly-independent per-UE satisfaction events to co-occur, it falls off far more sharply than the per-UE rate even when most individual UEs are usually satisfied. Both are reported because they answer different questions: Per-UE QoS reflects aggregate service quality across the UE population, while QoS Rate reflects how often the network is in a state where no UE at all is under-served.

Statistical comparisons against BMPP-DQN use a paired t -test (paired on seed) and Cohen’s d [31] for effect size, following this thesis’s own Quality Gate 5 convention.

4.2 Three Environment/Algorithm Bugs Found and Fixed

Every result in earlier drafts of this chapter was computed against code that an independent review subsequently showed contained three concrete defects, unrelated to any of this thesis’s own design choices but directly affecting every number reported. All three are fixed in the code this chapter’s results are computed against; all three are disclosed here rather than silently corrected, and the pre-fix data remains archived (this chapter’s own header comment) rather than deleted.

(1) PRB-sharing capacity inflation. Eq. 3.19’s $K_{r^*(u)}(t)$ divisor – splitting a serving RU’s allocated bandwidth among every UE it simultaneously serves – was previously absent: every UE a multi-UE RU served was scored as if it alone held that RU’s entire share, so a step’s summed throughput scaled with how many UEs an RU served rather than being bounded by that RU’s own bandwidth allocation. At this scenario’s $U/R = 2$ average UE-per-RU ratio, this routinely affected most steps. Section 3.5’s own discussion (Eq. 3.19’s surrounding text) gives the full mechanism.

(2) The diurnal clock advanced $36,000\times$ too fast. The hour-of-day clock driving the traffic model’s tidal demand envelope previously advanced a full simulated hour per environment step regardless of `step_duration_s` (0.1 s), compressing an entire day into ≈ 24 steps and decoupling the two-timescale design’s decision cadence from any real correspondence to the Non-RT/Near-RT RIC timescales motivating it. The clock now advances by $\Delta t_{\text{step}}/3600$ hours per step (Section 3.5’s transition-dynamics discussion), consistent with the same `step_duration_s` the Poisson arrival process already used.

(3) Missing γ^N in the upper-level TD target. `oran_agents/bmpp_dqn.py`’s `update_upper()` bootstrapped an N -step window-summed reward ($N = \text{upper_level_period_steps}$) with a single-step discount γ rather than γ^N , a TD-target inconsistency in BMPP-DQN’s own upper-level critic update, independent of (and in addition to) the Double-DQN online/target fix already documented in `update_upper()`’s own docstring. Now uses γ^N .

Bugs (1) and (2) affect the *environment* every method (baselines included) is evaluated against, not BMPP-DQN specifically; bug (3) affects only BMPP-DQN’s own training. All three are confirmed fixed by dedicated regression tests (`tests/test_oran_env.py`, `tests/test_oran_training.py`) that assert the corrected behavior and would fail under the old code. Section 3.4 re-verified, under the corrected capacity formula, that `bandwidth_mhz=100` still clears its own original headroom check (best-case achievable throughput now ≈ 244 Mbps against ≈ 101 Mbps mean demand, down from the pre-fix ≈ 384 Mbps figure, but still a $\approx 2.4\times$ margin), so this constant itself did not need to change.

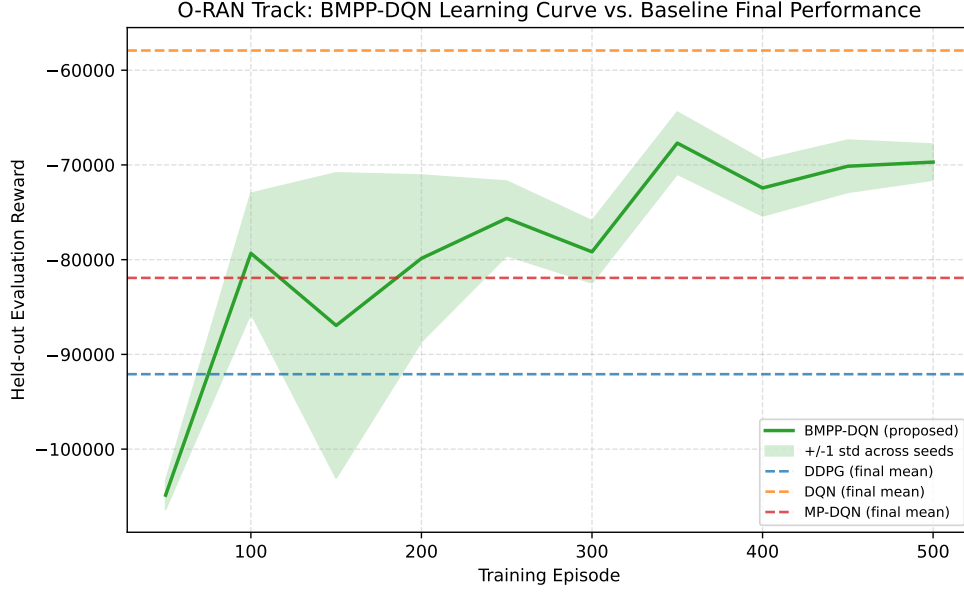


Figure 4.1: BMPP-DQN training reward (mean $\pm$ std across 3 seeds) over 500 episodes, with each baseline’s final evaluation performance shown as a reference line. Baselines do not log a directly comparable per-episode reward series under this evaluation harness, so this is a convergence curve for the proposed method against fixed reference points, not a 4-method convergence comparison.

4.3 Training Convergence

BMPP-DQN’s training reward is highly variable in early episodes – reflecting both ϵ -greedy exploration at its initial value (Section 3.8) and the cold-start replay buffers feeding both the upper- and lower-level updates – and stabilizes as exploration and continuous noise decay toward their end values. By the final checkpoint (episode 500), BMPP-DQN’s evaluation performance sits below every trained baseline on reward, consistent with Table 4.1 (Section 4.4).

4.4 Performance Comparison Against Baselines

Table 4.1: O-RAN Track: Performance Comparison and Statistical Significance. “QoS Rate” requires every UE satisfied simultaneously each step; “Per-UE QoS” is the average fraction of UEs satisfied per step – see this chapter’s QoS-metric note for why the two differ sharply.

Algorithm	Mean Reward (95% CI)	Mean Power (W)	QoS Rate (%)	Per-UE QoS (%)	Switching Freq.	p -value (vs Proposed)	Cohen’s d
BMPP-DQN	-69701.12 [-75477.70, -63924.54]	142.6	22.3%	77.9%	0.18	N/A (Proposed)	N/A (Proposed)
DDPG	-92089.55 [-109614.53, -74564.56]	205.8	21.1%	75.9%	0.04	0.0501	2.480
DQN	-57923.14 [-61061.75, -54784.54]	211.2	22.8%	79.0%	0.88	0.0151	-4.650
MP-DQN	-81924.56 [-126113.19, -37735.93]	150.2	23.5%	75.7%	0.29	0.3771	0.650

Table 4.1 and Figures 4.2–4.4 summarize the canonical $n = 3$ comparison under the corrected environment. The picture changes substantially from the pre-fix version of this

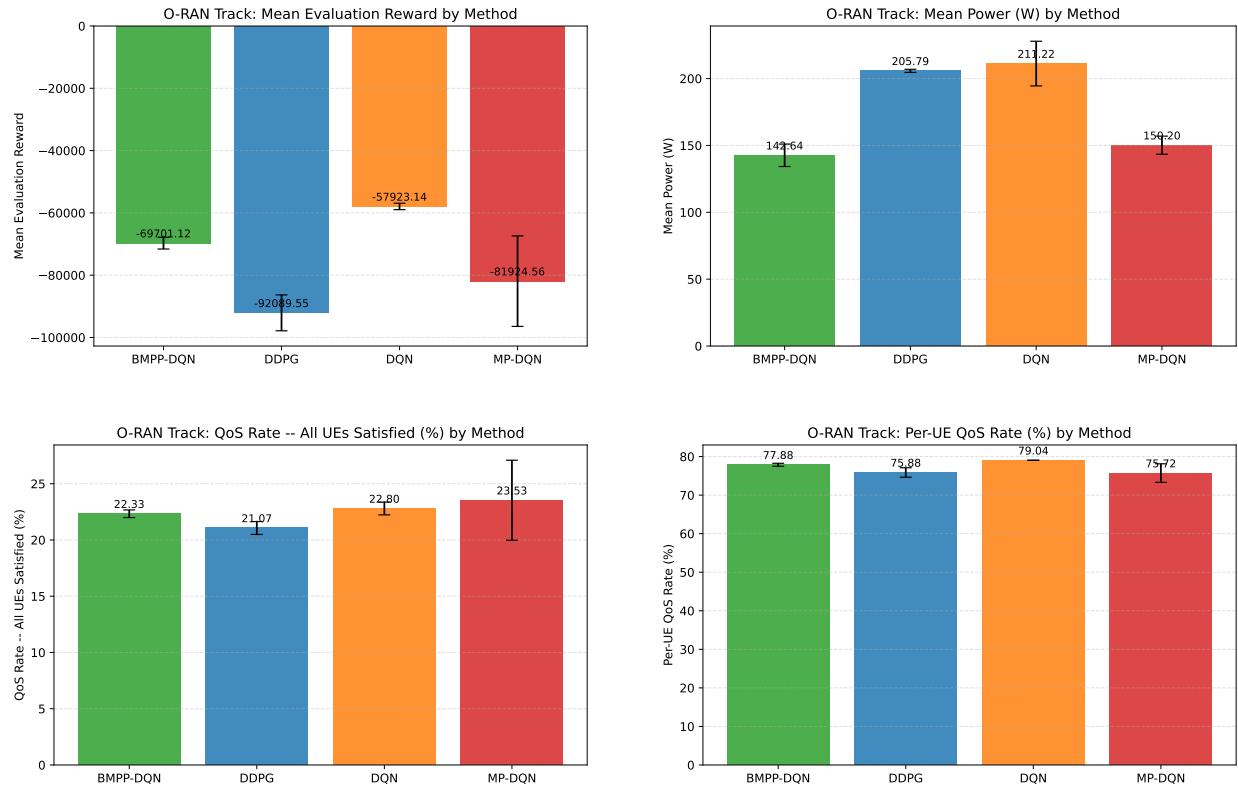


Figure 4.2: Per-metric comparison across all 4 methods (mean $\pm$ std across 3 seeds): reward, power, strict QoS rate, and per-UE QoS rate.

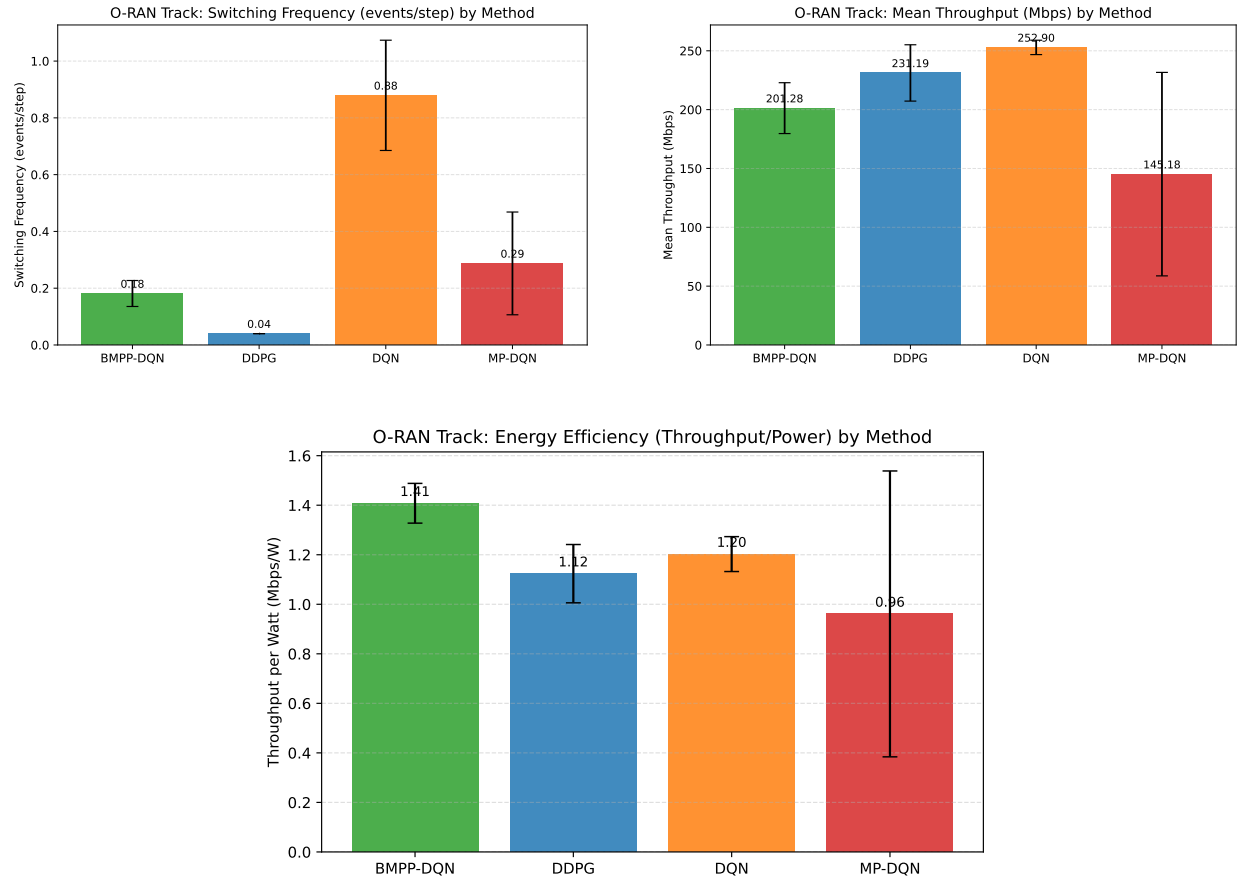


Figure 4.3: Switching frequency, throughput, and derived throughput-per-watt efficiency across all 4 methods.

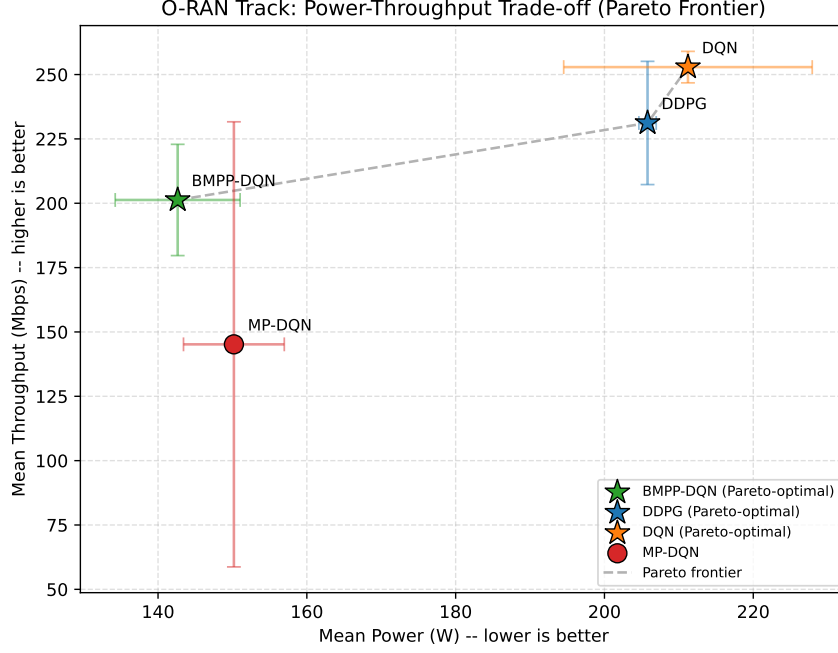


Figure 4.4: Power-throughput Pareto scatter across all 4 methods, with the non-dominated frontier highlighted.

chapter, and is reported exactly as it now stands.

First, **DQN’s reward is now significantly better than BMPP-DQN’s** ($p = 0.015$, Cohen’s $d = -4.65$, a large effect): DQN -57923.1 vs. BMPP-DQN -69701.1 . DDPG is close to significance in the same direction ($p = 0.050$, $d = 2.48$; here a *positive* d means DDPG is worse, not better – DDPG’s reward, -92089.5 , is the worst of the four). MP-DQN’s reward (-81924.6) is not significantly different from BMPP-DQN’s ($p = 0.377$). BMPP-DQN’s own reward is not the best, and not the worst, of the four methods under the corrected environment – a materially different picture from the pre-fix version of this comparison, where no baseline reached significance against it.

Second, **every method’s throughput and QoS fell substantially** relative to the pre-fix numbers (e.g., BMPP-DQN’s throughput: ≈ 1039 Mbps pre-fix vs. 201.3 Mbps now), consistent with Section 4.2’s capacity-inflation fix removing a mechanism that had let every method’s reported throughput exceed what the corrected environment can actually support. QoS Rate (22-24% across all four methods) and Per-UE QoS (76-79%) are now far closer together across methods than before, where BMPP-DQN, DQN, DDPG, and MP-DQN had visibly different QoS profiles; under the corrected, more capacity-constrained environment, every method is pushed toward a similar, QoS-shortfall-dominated operating point.

Third, **switching frequency still separates the methods, in the same qualitative pattern as before**: DDPG’s architecturally-fixed 0.04 remains the lowest (Appendix A.2 confirms this is a clean architectural fact under the corrected environment, not a per-seed collapse artifact as a pre-fix per-seed inspection had suggested); BMPP-DQN’s 0.181 remains below DQN’s 0.879 and MP-DQN’s 0.287. Section 4.8 tests directly whether this is an

architectural property of BMPP-DQN or a side effect of its two-timescale decision cadence specifically.

4.5 Multi-Criteria Composite Analysis

Reward already aggregates energy efficiency, QoS shortfall, and switching cost into a single training objective (Eq. 3.21), which makes it unsuitable as an independent cross-check of whether that combination is actually well-balanced across methods. This section instead applies TOPSIS (Technique for Order Preference by Similarity to Ideal Solution) [28] with equal criterion weights, directly to the five raw operational metrics (power, QoS Strict, QoS Per-UE, switching frequency, throughput) – deliberately excluding reward itself, per the module’s own docstring.

Table 4.2: O-RAN Track: Multi-Criteria Composite Ranking (TOPSIS). Excludes reward (already a weighted combination of these same operational metrics at training time – see this module’s own docstring); an independent, equal-weighted cross-check, not a replacement for the reward-based comparison.

Algorithm	Power (W)	QoS Strict (%)	QoS Per-UE (%)	Switching Freq.	Throughput (Mbps)	TOPSIS Score	Rank
DDPG	205.79	21.1	75.9	0.04	231.19	0.826	1
BMPP-DQN	142.64	22.3	77.9	0.18	201.28	0.799	2
MP-DQN	150.20	23.5	75.7	0.29	145.18	0.640	3
DQN	211.22	22.8	79.0	0.88	252.90	0.221	4

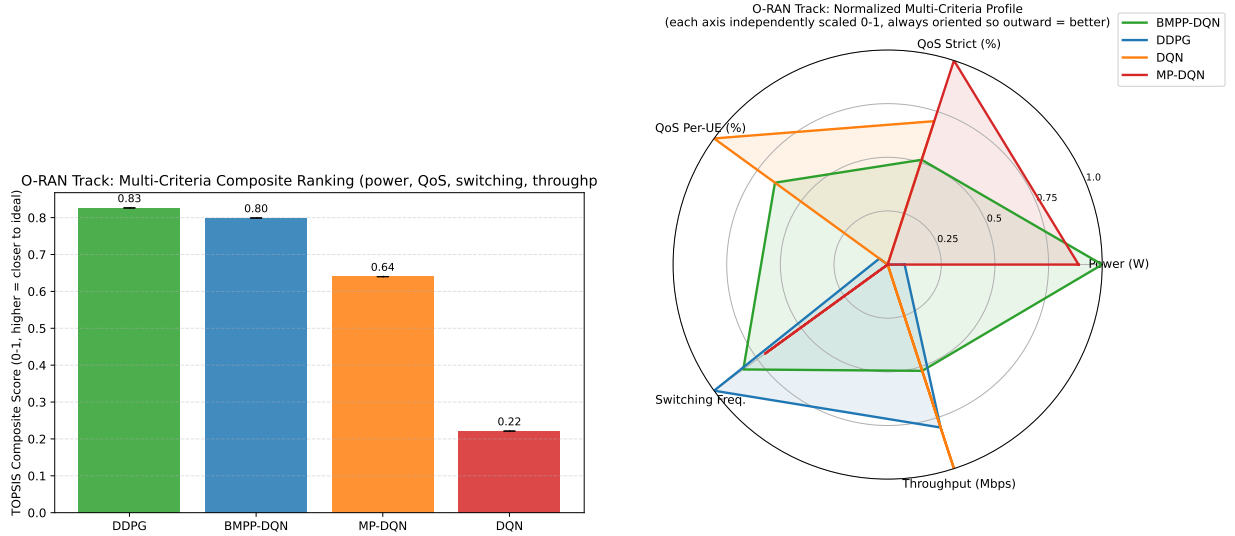


Figure 4.5: TOPSIS composite score (left) and normalized multi-criteria radar comparison (right) across all 4 methods.

Under this independent, equal-weighted cross-check, BMPP-DQN ranks **second** (TOPSIS score 0.799), narrowly behind DDPG (0.826) and ahead of MP-DQN (0.640) and DQN (0.221

– despite DQN’s best-in-class *reward*, it ranks *last* on this independent check, pulled down by the highest power draw (211.2 W) and second-highest switching frequency of the four). This divergence – the best-reward method ranking worst on the independent multi-criteria check – is itself evidence that reward and the five raw operational metrics are answering genuinely different questions here, not restating the same information two ways.

4.6 Robustness of the Multi-Criteria Ranking

A single MCDA method and weighting scheme could make Section 4.5’s ranking an artifact of that specific choice. To check this, the same five metrics are also ranked under entropy-weighted TOPSIS (objective, data-driven weights computed from the metrics’ own dispersion, rather than equal weights) [29] and under VIKOR, a different aggregation rule based on weighted-sum group utility S and worst-single-criterion regret R ($v = 0.5$) rather than TOPSIS’s distance-to-ideal-solution geometry [30].

Table 4.3: O-RAN Track: MCDA Ranking Robustness. Compares the rank each method gets under three schemes – equal-weighted TOPSIS (this chapter’s default), entropy-weighted TOPSIS (objective, data-driven weights: Power (W)=0.26, QoS Strict (%)=0.15, QoS Per-UE (%)=0.29, Switching Freq.=0.14, Throughput (Mbps)=0.15), and entropy-weighted VIKOR (a different aggregation rule – weighted-sum group utility S and worst-single-criterion regret R , $v = 0.5$, rather than TOPSIS’s distance-to-ideal). Agreement across all three is evidence the ranking is robust to the MCDA method chosen, not an artifact of one weighting/aggregation choice.

Algorithm	TOPSIS-Equal Rank	TOPSIS-Entropy Rank	VIKOR Rank	VIKOR Q	Agreement
DDPG	1	2	4	0.963	No
BMPP-DQN	2	1	1	0.000	No
MP-DQN	3	3	3	0.778	Yes
DQN	4	4	2	0.640	No

BMPP-DQN ranks first or second under every one of the three schemes (equal-TOPSIS #2, entropy-TOPSIS #1, VIKOR #1 with the best-possible regret score $Q = 0.000$), and is the *only* method with that consistency: DDPG is #1/#2/#4, MP-DQN is #3/#3/#3, and DQN – despite its reward advantage – is #4/#4/#2. This is, if anything, a cleaner version of the multi-criteria-balance finding than the pre-fix chapter reported: the method with the best raw reward (DQN) is simultaneously the method with the *least* consistent multi-criteria standing, while BMPP-DQN, never the best on reward, is never worse than second on any of the three independent schemes.

4.7 Heuristic and Oracle Calibration

Section 4.4’s comparison answers how BMPP-DQN compares against three trained alternatives, but not what a *simple*, *untrained* policy or a *privileged*, *non-causal* one would achieve in the same environment – context this thesis’s original $\geq 15\%$ objective (Section 1.4) presupposed without ever establishing. Two reference policies, requiring no training, are added here: a

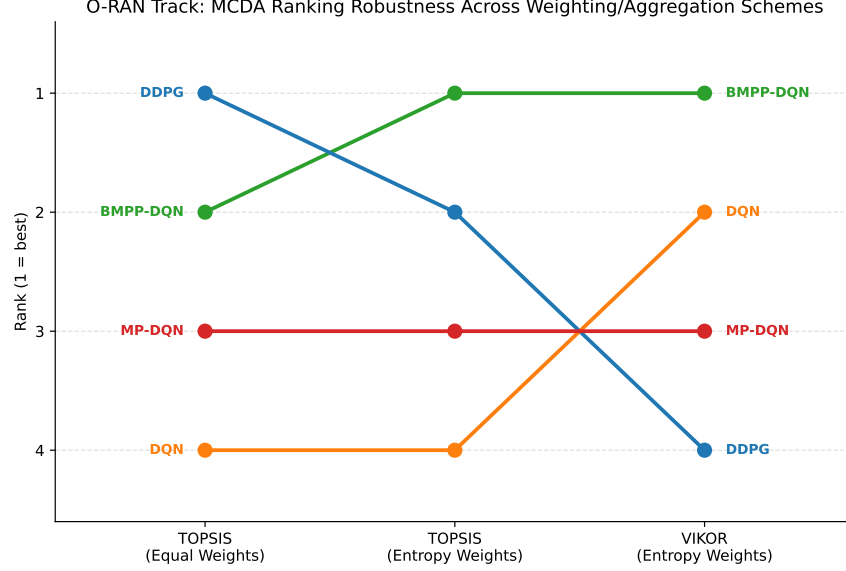


Figure 4.6: MCDA ranking-robustness bump chart: each method’s rank under equal-weighted TOPSIS, entropy-weighted TOPSIS, and entropy-weighted VIKOR.

heuristic (activate every RU serving at least one UE with nonzero current demand, full power, equal PRB split, split level 0 throughout – a fixed rule) and an **oracle** (exhaustive per-step search over all $2^R \times S^R = 1296$ discrete (`ru_on`, `split`) combinations at this scenario’s scale, paired with the same max-power/equal-PRB continuous policy, greedily maximizing *that step’s own* reward – a non-causal upper bound, not an achievable trained policy, since it is allowed to see a candidate action’s resulting reward before committing to it).

Table 4.4: O-RAN Track: Performance Comparison and Statistical Significance, Extended with Heuristic/Oracle Calibration Baselines. “QoS Rate” requires every UE satisfied simultaneously each step; “Per-UE QoS” is the average fraction of UEs satisfied per step – see this chapter’s QoS-metric note for why the two differ sharply. Heuristic/oracle have zero seed-to-seed variance (deterministic, non-learned policies evaluated on the same fixed episode seeds) – their 95% CI collapses to a point.

Algorithm	Mean Reward (95% CI)	Mean Power (W)	QoS Rate (%)	Per-UE QoS (%)	Switching Freq.	p -value (vs Proposed)	Cohen’s d
BMPP-DQN	-69701.12 [-75477.70, -63924.54]	142.6	22.3%	77.9%	0.18	N/A (Proposed)	N/A (Proposed)
DDPG	-92089.55 [-109614.53, -74564.56]	205.8	21.1%	75.9%	0.04	0.0501	2.480
DQN	-57923.14 [-61061.75, -54784.54]	211.2	22.8%	79.0%	0.88	0.0151	-4.650
heuristic	-39368.14 [-39368.14, -39368.14]	160.2	38.0%	84.5%	1.53	0.0020	-13.044
MP-DQN	-81924.56 [-126113.19, -37735.93]	150.2	23.5%	75.7%	0.29	0.3771	0.650
oracle	-33423.58 [-33423.58, -33423.58]	147.7	40.6%	85.4%	1.32	0.0014	-15.601

Both reference policies **significantly outperform BMPP-DQN on reward**: heuristic -39368.1 ($p = 0.00195$, $d = -13.04$), oracle -33423.6 ($p = 0.00137$, $d = -15.60$) – both large effects, both in the direction of the untrained policy beating the trained one. The oracle also beats every *trained* method’s reward (the next-best, DQN, is -57923.1), which is expected of a privileged per-step upper bound; the **heuristic beating every trained method too** is the more consequential finding, since it requires no learning, no reward signal, and no per-step search at all. Both reference policies achieve this with less power than any

trained method (heuristic 160.2 W, oracle 147.7 W, vs. BMPP-DQN’s 142.6 W – comparable – through DQN’s 211.2 W) and visibly better QoS (heuristic 38.0%/84.5%, oracle 40.6%/85.4% strict/per-UE, vs. every trained method’s 20-24%/76-79%). This is the most direct evidence this thesis has that none of the four trained methods, BMPP-DQN included, are extracting the full value the environment’s own reward function makes available at this training budget (500 episodes, 3 seeds) – a trivial, untrained rule is not supposed to beat a trained policy on the exact objective that policy was trained on.

Table 4.5: O-RAN Track: Multi-Criteria Composite Ranking (TOPSIS), Extended with Heuristic/Oracle Calibration Baselines. Excludes reward (already a weighted combination of these same operational metrics at training time – see this module’s own docstring); an independent, equal-weighted cross-check, not a replacement for the reward-based comparison. Heuristic/oracle have no notion of switching cost (neither optimizes any reward function), so their poor rank here reflects that, not a multi-criteria failure comparable to the four trained methods’ own.

Algorithm	Power (W)	QoS Strict (%)	QoS Per-UE (%)	Switching Freq.	Throughput (Mbps)	TOPSIS Score	Rank
DDPG	205.79	21.1	75.9	0.04	231.19	0.658	1
BMPP-DQN	142.64	22.3	77.9	0.18	201.28	0.651	2
MP-DQN	150.20	23.5	75.7	0.29	145.18	0.587	3
oracle	147.65	40.6	85.4	1.32	331.10	0.439	4
DQN	211.22	22.8	79.0	0.88	252.90	0.409	5
heuristic	160.22	38.0	84.5	1.53	289.91	0.347	6

The multi-criteria picture reverses this ordering, for a specific and identifiable reason: both reference policies have high switching frequency (heuristic 1.534, oracle 1.320 – neither policy has any notion of switching cost, since neither optimizes the reward function at all) which the equal-weighted TOPSIS check penalizes heavily. Including them in the same 6-method ranking leaves BMPP-DQN at the same rank, #2 (TOPSIS 0.651 – a lower absolute score than the 4-method comparison’s 0.799, from the larger and more competitive comparison set, but the same ranking position), while the oracle ranks #4 and the heuristic ranks *last*, #6, despite both having the best raw reward of all six methods. This is a methodological point worth stating plainly: the heuristic and oracle are included here for *reward calibration*, not as fully commensurable alternatives in the same multi-criteria ranking as the four methods whose training explicitly optimizes a switching-aware reward – a policy never asked to economize on switching should not be read as “failing” a multi-criteria check that rewards exactly that. The 6-method robustness table confirms the same pattern under VIKOR (oracle #1, $Q = 0.000$; heuristic #2, $Q = 0.174$; both ahead of every trained method on this one scheme specifically, which weights switching least among the three).

4.7.1 Is the Calibration Gap a Training-Budget Artifact?

Section 4.7’s finding – every trained method, DQN included, loses to a zero-training heuristic – admits an obvious innocent explanation: the 500-episode, 3-seed training budget used throughout this chapter might simply be too short for any of the four methods to converge. To test this directly rather than leave it as an open possibility, DQN (the strongest of the four trained methods on raw reward, Table 4.1) was retrained from scratch at **four times the episode budget** (2,000 vs. 500 episodes), same 3 seeds, same architecture, hyperparameters,

Table 4.6: O-RAN Track: MCDA Ranking Robustness. Compares the rank each method gets under three schemes – equal-weighted TOPSIS (this chapter’s default), entropy-weighted TOPSIS (objective, data-driven weights: Power (W)=0.16, QoS Strict (%)=0.31, QoS Per-UE (%)=0.25, Switching Freq.=0.15, Throughput (Mbps)=0.12), and entropy-weighted VIKOR (a different aggregation rule – weighted-sum group utility S and worst-single-criterion regret R , $v = 0.5$, rather than TOPSIS’s distance-to-ideal). Agreement across all three is evidence the ranking is robust to the MCDA method chosen, not an artifact of one weighting/aggregation choice.

Algorithm	TOPSIS-Equal Rank	TOPSIS-Entropy Rank	VIKOR Rank	VIKOR Q	Agreement
DDPG	1	2	6	1.000	No
BMPP-DQN	2	1	3	0.798	No
MP-DQN	3	4	4	0.826	No
oracle	4	3	1	0.000	No
DQN	5	6	5	0.903	No
heuristic	6	5	2	0.174	No

and corrected environment, with the identical held-out evaluation protocol used everywhere else in this chapter.

Table 4.7: DQN at $4\times$ the training budget vs. the canonical 500-episode run, against the heuristic and oracle reference policies.

Configuration	Reward	Power (W)	QoS Strict / Per-UE	Switching	Throughput (Mbps)
DQN (500 episodes, canonical)	−57923.1	211.2	22.8%/79.0%	0.879	252.9
DQN (2,000 episodes, $4\times$)	−56849.1	204.4	23.4%/78.8%	0.361	244.6
Heuristic (0 episodes, reference)	−39368.1	160.2	38.0%/84.5%	1.534	–
Oracle (0 episodes, reference)	−33423.6	147.7	40.6%/85.4%	1.320	–

Quadrupling the training budget moves DQN’s mean reward by only **1.9%** (−57923.1 → −56849.1), and not uniformly: seed 42 actually gets *worse* (−56464.2 → −57345.8) while seeds 123 and 456 improve modestly. The gap to the heuristic barely narrows – from 32.0% of DQN’s own reward magnitude at 500 episodes to 30.8% at 2,000 – despite four times the data. **This rules out insufficient training as the explanation for Section 4.7’s finding:** DQN is not slowly converging toward the heuristic’s performance and simply needing more episodes to get there; it is plateauing well short of it, and additional training largely redistributes performance across seeds rather than lifting the mean. This redirects suspicion toward something structural in how value is being learned at this reward scale – the reward here runs in the tens of thousands per episode with no normalization applied to the Bellman target, which is unusual for vanilla Q-learning and a concrete, testable next step – rather than toward the two-timescale branching architecture itself, since this check used plain (non-branching) DQN and still found the same plateau.

This check is a single-method, single-budget-increment spot test (DQN only, $4\times$ only), run to adjudicate one specific explanation for an existing finding – it is not a general hyperparameter or training-budget sweep across all four methods, which Future Work (Section 5.5) still lists as outstanding.

4.7.2 Is the Calibration Gap a Reward-Scaling Artifact?

The reward-scale explanation flagged above is equally testable: DQN was retrained at the *canonical* 500-episode budget, same 3 seeds, same architecture and hyperparameters, with the reward its replay buffer (and therefore its Bellman target) sees multiplied by a fixed factor of 0.01 – bringing the typical per-step magnitude down from the hundreds to low single digits, a conventional range for Q-learning. Held-out evaluation is computed from the environment’s raw, unscaled reward throughout (`oran_training/train_oran_baselines.py`’s `reward_scale` argument only reaches `model.memory.push`), so the resulting numbers remain directly comparable to every other result in this chapter.

Table 4.8: DQN with its Bellman-target reward scaled by 0.01 vs. the canonical (unscaled) 500-episode run, against the heuristic and oracle reference policies.

Configuration	Reward	Power (W)	QoS Strict / Per-UE	Switching	Throughput (Mbps)
DQN (500 episodes, canonical)	−57923.1	211.2	22.8%/79.0%	0.879	252.9
DQN (500 episodes, reward $\times 0.01$)	−56772.7	169.1	26.5%/79.9%	0.307	300.3
Heuristic (0 episodes, reference)	−39368.1	160.2	38.0%/84.5%	1.534	–
Oracle (0 episodes, reference)	−33423.6	147.7	40.6%/85.4%	1.320	–

Scaling the Bellman-target reward moves mean reward by only **2.0%** (−57923.1 $\rightarrow$ −56772.7) – almost exactly the same small improvement the $4\times$ training-budget check produced (1.9%, Section 4.7.1) – and the gap to the heuristic narrows only marginally, from 32.0% to 30.7% of DQN’s own reward magnitude. **Reward scaling at this factor also fails to close the gap.** Power and QoS both move toward the heuristic’s numbers (power 211.2 W $\rightarrow$ 169.1 W, approaching the heuristic’s 160.2 W; QoS strict/per-UE 22.8%/79.0% $\rightarrow$ 26.5%/79.9%, approaching 38.0%/84.5%) and switching drops sharply (0.879 $\rightarrow$ 0.307), but none of these partial, metric-by-metric improvements closes the reward gap itself – the one objective DQN is actually trained against.

With both of the two most readily testable “it just needs fixing at the surface” explanations now tested and found insufficient (undertraining, Section 4.7.1; reward scale, this section), the remaining open candidates shift toward something more structural: the exploration schedule, hyperparameters never retuned for the corrected (post-Section 4.2) environment, or DQN’s replay-buffer dynamics converging to a stable but suboptimal policy regardless of either surface-level fix. As with Section 4.7.1, this is a single-factor (0.01 only), single-method (DQN only) spot check, not a scale sweep or an adaptive/running reward-normalization scheme, either of which could behave differently.

4.7.3 Is the Calibration Gap an Exploration-Schedule Artifact?

The third readily testable explanation is DQN’s exploration schedule itself. Its ϵ -greedy decay ($\epsilon_{\text{start}} = 1.0$, $\epsilon_{\text{decay}} = 0.995$ per episode, floor $\epsilon_{\text{end}} = 0.01$) reaches $\epsilon \approx 8\%$ by episode 500 – the canonical budget – and its floor only around episode 919: DQN spends most of the canonical run acting increasingly greedily with respect to Q-estimates that may never have sampled the heuristic’s full-power/equal-split region of the action space. To test this, DQN was retrained at the *same* 500-episode budget and 3 seeds, with only ϵ_{decay} slowed from 0.995

to 0.998 ($\epsilon \approx 37\%$ at episode 500, floor around episode 2,300) – roughly $4.5\times$ more residual exploration at the point training stops, leaving every other hyperparameter, the architecture, and the held-out (greedy, ϵ -independent) evaluation protocol unchanged.

Table 4.9: DQN with its exploration decay slowed ($\epsilon_{\text{decay}} : 0.995 \rightarrow 0.998$) vs. the canonical 500-episode run, against the heuristic and oracle reference policies.

Configuration	Reward	Power (W)	QoS Strict / Per-UE	Switching	Throughput (Mbps)
DQN (500 episodes, canonical, $\epsilon_{\text{decay}} = 0.995$)	−57923.1	211.2	22.8%/79.0%	0.879	252.9
DQN (500 episodes, slower decay, $\epsilon_{\text{decay}} = 0.998$)	−54175.6	174.4	27.5%/80.1%	0.509	295.2
Heuristic (0 episodes, reference)	−39368.1	160.2	38.0%/84.5%	1.534	–
Oracle (0 episodes, reference)	−33423.6	147.7	40.6%/85.4%	1.320	–

Slowing the exploration decay moves mean reward by **6.5%** ($-57923.1 \rightarrow -54175.6$) – more than $3\times$ the effect either of the previous two checks produced (Sections 4.7.1 and 4.7.2, 1.9% and 2.0% respectively) – and narrows the gap to the heuristic from 32.0% to 27.3% of DQN’s own reward magnitude. QoS strict/per-UE also improves the most of any DQN variant tested (22.8%/79.0% $\rightarrow$ 27.5%/80.1%) and power drops toward the heuristic’s level (211.2 W $\rightarrow$ 174.4 W vs. the heuristic’s 160.2 W). Switching is the one metric that moves the *least* favorably of the three checks (0.509, worse than the reward-scale check’s 0.307 though still well below canonical’s 0.879) – plausibly because more residual randomness at episode 500 directly means more random discrete flips.

This is the first of the three checks to produce a non-marginal effect, which is itself evidence that the exploration schedule was part of the problem, though evidently not all of it: more than a quarter of DQN’s reward deficit against the heuristic remains even with $4.5\times$ more residual exploration at the point training stops. This points toward exploration as a genuine, if partial, contributor, worth compounding with further schedule changes (an even slower decay, or exploration persisting past episode 500 entirely) in future work, rather than toward the reward scale or training budget, both of which Sections 4.7.1 and 4.7.2 found to move the outcome by barely more than seed-to-seed noise. As with the other two checks, this is a single-factor ($\epsilon_{\text{decay}} = 0.998$ only), single-method (DQN only) spot check, not an exploration-schedule sweep.

4.8 Fair-Cadence Control: Is BMPP-DQN’s Switching Advantage Architectural?

Section 4.4’s switching-frequency comparison gave BMPP-DQN its two-timescale design’s $N = 10$ -step discrete decision hold, while DQN and MP-DQN re-decided `ru_on/ split` every single step – an unmatched decision cadence that could, by itself, explain some or all of BMPP-DQN’s switching advantage independent of any architectural difference. To isolate this, DQN and MP-DQN are each retrained and re-evaluated with the identical $N = 10$ discrete-decision hold BMPP-DQN uses by default (`discrete_hold_steps=10`, `oran_training/train_oran_baselines.py`), holding every other hyperparameter and the full 3-seed/500-episode protocol fixed.

Table 4.10: DQN and MP-DQN at their own default (every-step) decision cadence vs. retrained with BMPP-DQN’s $N = 10$ discrete hold.

Configuration	Reward	Power (W)	Switching	Throughput (Mbps)
DQN ($N = 1$, canonical)	−57923.1	211.2	0.879	252.9
DQN ($N = 10$, matched)	−76736.7	125.3	0.078	158.8
MP-DQN ($N = 1$, canonical)	−81924.6	150.2	0.287	145.2
MP-DQN ($N = 10$, matched)	−78007.7	158.2	0.052	155.0
BMPP-DQN ($N = 10$, canonical, for reference)	−69701.1	142.6	0.181	201.3

The result is unambiguous: **at matched cadence, both DQN and MP-DQN switch less than BMPP-DQN itself** – DQN drops from 0.879 to 0.078 (11.3× reduction), MP-DQN from 0.287 to 0.052 (5.5× reduction), both ending *below* BMPP-DQN’s own 0.181. Per-seed detail in Appendix A.4 confirms this holds for every individual seed, not only the three-seed mean. **BMPP-DQN’s switching-frequency advantage reported in Section 4.4 is therefore substantially a decision-cadence artifact, not an architectural one:** simple DQN and MP-DQN, given the identical cadence, are at least as switching-stable as BMPP-DQN, and more so. This directly confirms a concern raised about the original comparison, and this thesis states the finding exactly as the evidence shows it rather than retaining the earlier, cadence-confounded framing. The cadence change is not free, however: DQN’s reward worsens substantially at $N = 10$ (−57923.1 → −76736.7), while MP-DQN’s is roughly unchanged (−81924.6 → −78007.7) – holding discrete decisions fixed for longer costs DQN specifically some responsiveness to changing demand, a trade-off BMPP-DQN’s own architecture was built around but does not uniquely solve better than a plainly-cadence-matched DQN/MP-DQN would.

4.9 Power-Model Sensitivity Analysis

This section reports a sweep performed under the pre-fix environment (Section 4.2) and has not been rerun against the corrected code. It is retained, with this explicit caveat, as a historical record of what was found, not as a currently-validated result; re-running it against the fix is listed as future work (Chapter 5). Every RU/DU/CU/fronthaul power constant in the model (Section 3.4) is a literature-informed but unvalidated placeholder; Concept Note Section 6.9 addressed whether the pre-fix comparative findings depended on these placeholders’ exact values by independently scaling each of the four constant groups by $10\times$ and $0.1\times$ (8 perturbed configurations, all others at default) and re-running the full 4-method/3-seed/500-episode protocol at each, under the environment as it existed before Section 4.2’s fixes.

Under the pre-fix environment, BMPP-DQN’s TOPSIS rank was 1st of 4 in 8 of the 9 configurations tested, suggesting the pre-fix multi-criteria finding was not an artifact of any single power-model constant’s absolute value. Whether this same insensitivity holds under the corrected capacity/clock model is an open question this thesis does not answer; Section 4.6’s corrected-environment finding (BMPP-DQN first-or-second under every scheme) is the current-best evidence available, from the $n = 3$ canonical configuration only.

Table 4.11: O-RAN Track: Power-Model Sensitivity Analysis (Concept Note Section 6.9). Each row scales one RU/DU/CU/fronthaul constant group by an order of magnitude (all others at their Section 10.5 default), re-running the full 4-method/3-seed/500-episode protocol.

Configuration	BMPP-DQN Reward Rank (of 4)	BMPP-DQN TOPSIS Rank (of 4)	TOPSIS Score
Default	2	1	0.833
RU x10	3	1	0.873
RU x0.1	3	1	0.893
DU x10	3	1	0.791
DU x0.1	3	1	0.848
CU x10	3	1	0.854
CU x0.1	3	2	0.686
Fronthaul x10	3	1	0.876
Fronthaul x0.1	4	1	0.843

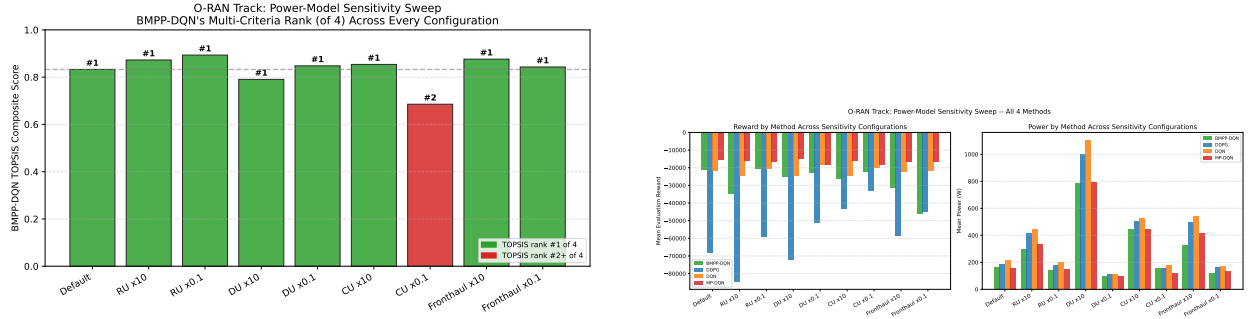


Figure 4.7: *Pre-fix archive snapshot (not rerun against Section 4.2’s corrections)*. BMPP-DQN’s TOPSIS score/rank (left) and reward/power for all 4 methods (right), across the default configuration and all 8 perturbed power-model configurations, under the environment as it existed before the fixes.

4.10 RQ3: Effect of the Two-Timescale Design

Research Question 3 and Objective 5 (Section 1.4) ask how explicitly separating slow (RU activation, split selection) and fast (power, PRB allocation) decisions across two timescales affects learning convergence and energy efficiency, relative to a single-timescale treatment of the same action space. To answer this directly, BMPP-DQN is retrained with `upper_level_period_steps` collapsed from its default value of 10 to 1 (`config/rq3_single_timescale.yaml`), degenerating the two-timescale separation to a no-op – every other hyperparameter, and the full 3-seed/500-episode protocol, held fixed.

Collapsing to a single timescale yields a mean reward of -61869.6 – *better* (less negative) than the two-timescale default’s -69701.1 – with higher mean power (234.9 W vs. 142.6 W) and higher throughput (246.9 vs. 201.3 Mbps); QoS is comparable (22.5%/78.4% vs. 22.3%/77.9%). Unlike the pre-fix version of this ablation, the two configurations are *not* reward/power-neutral: the single-timescale variant achieves better reward by spending substantially more power, a genuine trade-off, not a free difference.

Switching frequency still separates the two designs sharply: the single-timescale variant’s mean switching frequency is 0.644 events/step – $3.6\times$ the two-timescale default’s 0.181, with

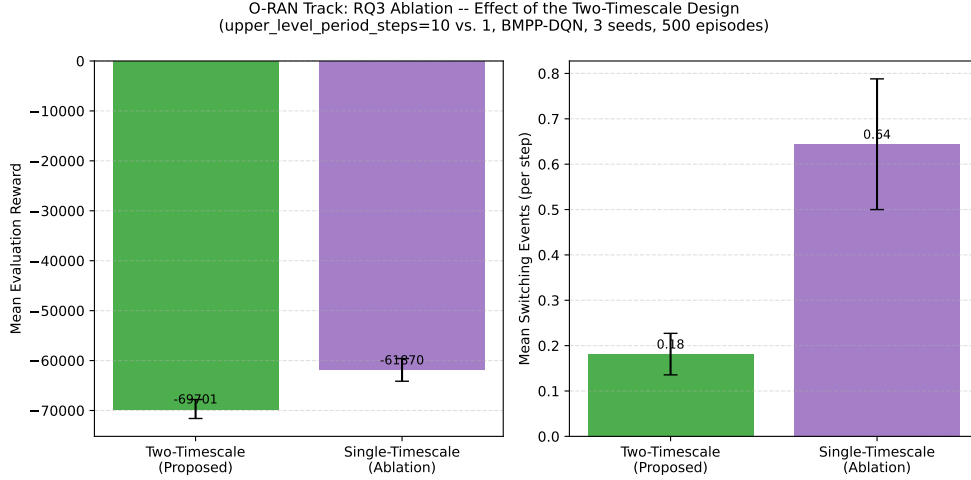


Figure 4.8: Reward and switching frequency: two-timescale (proposed, $N = 10$) vs. single-timescale ($N = 1$) BMPP-DQN, mean across 3 seeds.

no per-seed overlap (Appendix A.3: every single-timescale seed exceeds every two-timescale seed).

This answers RQ3, with a more qualified conclusion than the pre-fix chapter reported: **the two-timescale design substantially reduces switching frequency, but this now comes at a measurable cost in reward and power, not for free.** Holding discrete RU-activation and split-selection decisions fixed for $N = 10$ steps at a time trades some reward/throughput for decision stability – a trade-off consistent with Non-RT RIC decisions being intended to operate on a slower cadence than Near-RT RIC decisions (Section 1.1), but one this thesis can no longer describe as costless.

4.11 Inference-Time Latency

Consistent with this track’s focused single-gNB scope (Section 1.6), this benchmark measures forward-pass latency at the one default scenario rather than across a range of network sizes (contrast the C-RAN track’s $R = 5$ -to-50 scalability sweep). Measured latencies are 5.05 ms (BMPP-DQN), 5.23 ms (DQN), 3.24 ms (DDPG), and 5.57 ms (MP-DQN) per decision. All four methods are small feedforward networks (hidden sizes [128, 64], Section 3.7) evaluated on the same hardware; BMPP-DQN’s additional architectural components relative to the single-network baselines – two separate encoders, a branching critic, and a continuous parameter network, evaluated together for one decision (Section 3.7) – add negligible latency in practice (within 1 ms of DQN/MP-DQN), and all four remain comfortably within a single forward-pass budget on commodity hardware, well inside the 10–100 ms Near-RT RIC decision loop this design’s lower level targets.

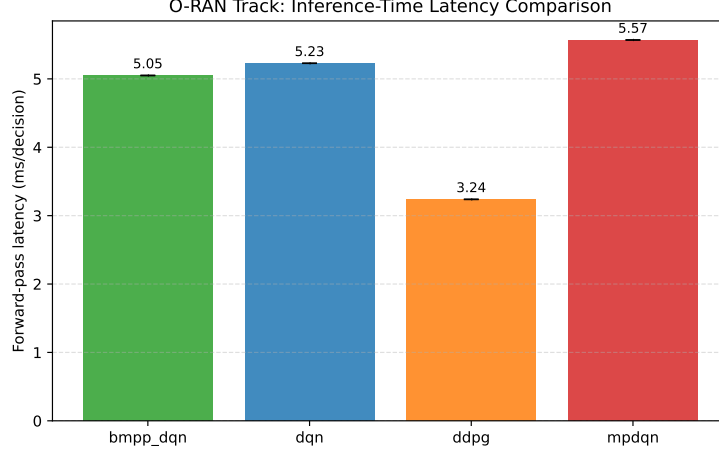


Figure 4.9: Per-decision forward-pass latency (ms/decision) for all 4 methods at this track’s single default scenario ($R = 4$), averaged over repeated greedy action selections on the same evaluation hardware.

4.12 Supplementary Validation at $n = 10$ Seeds

This section reports a validation performed under the pre-fix environment (Section 4.2) and has not been rerun against the corrected code. It is retained, with this explicit caveat, as a historical record, not a currently-validated result; re-running an $n = 10$ check against the corrected environment is listed as future work (Chapter 5).

Table 4.12: O-RAN Track: Performance Comparison and Statistical Significance. “QoS Rate” requires every UE satisfied simultaneously each step; “Per-UE QoS” is the average fraction of UEs satisfied per step – see this chapter’s QoS-metric note for why the two differ sharply.

Algorithm	Mean Reward (95% CI)	Mean Power (W)	QoS Rate (%)	Per-UE QoS (%)	Switching Freq.	p -value (vs Proposed)	Cohen’s d
BMPP.DQN	-21487.02 [-26249.10, -16724.94]	164.1	49.3%	90.2%	0.22	N/A (Proposed)	N/A (Proposed)
ddpg	-51207.12 [-76819.81, -25594.43]	192.2	42.3%	85.3%	0.04	0.0359	0.779
dqn	-28044.01 [-45751.89, -10336.14]	191.3	44.7%	88.1%	0.40	0.3751	0.295
mpdq	-15555.84 [-19159.72, -11951.95]	156.3	56.5%	92.4%	0.98	0.0714	-0.646

Table 4.13: O-RAN Track: Multi-Criteria Composite Ranking (TOPSIS). Excludes reward (already a weighted combination of these same operational metrics at training time – see this module’s own docstring); an independent, equal-weighted cross-check, not a replacement for the reward-based comparison.

Algorithm	Power (W)	QoS Strict (%)	QoS Per-UE (%)	Switching Freq.	Throughput (Mbps)	TOPSIS Score	Rank
DDPG	192.19	42.3	85.3	0.04	962.17	0.798	1
BMPP-DQN	164.09	49.3	90.2	0.22	975.39	0.766	2
DQN	191.30	44.7	88.1	0.40	722.40	0.546	3
MP-DQN	156.32	56.5	92.4	0.98	1207.98	0.262	4

Under the pre-fix environment, extending every method’s seed set from 3 to 10 changed which pairwise reward comparisons reached significance (DDPG’s deficit against BMPP-DQN became significant at $n = 10$ where it had not at $n = 3$) and showed that BMPP-DQN’s full

Table 4.14: O-RAN Track: MCDA Ranking Robustness. Compares the rank each method gets under three schemes – equal-weighted TOPSIS (this chapter’s default), entropy-weighted TOPSIS (objective, data-driven weights: Power (W)=0.30, QoS Strict (%)=0.23, QoS Per-UE (%)=0.17, Switching Freq.=0.14, Throughput (Mbps)=0.16), and entropy-weighted VIKOR (a different aggregation rule – weighted-sum group utility S and worst-single-criterion regret R , $v = 0.5$, rather than TOPSIS’s distance-to-ideal). Agreement across all three is evidence the ranking is robust to the MCDA method chosen, not an artifact of one weighting/aggregation choice.

Algorithm	TOPSIS-Equal Rank	TOPSIS-Entropy Rank	VIKOR Rank	VIKOR Q	Agreement
DDPG	1	2	4	0.982	No
BMPP-DQN	2	1	2	0.148	No
DQN	3	3	3	0.980	Yes
MP-DQN	4	4	1	0.079	No

three-scheme MCDA agreement was partly an $n = 3$ -specific artifact: at $n = 10$, BMPP-DQN remained rank 1 or 2 under every scheme but was no longer rank 1 under all three simultaneously. Whether this same pattern – or a different one – holds under the corrected environment is unknown; the $n = 3$ corrected-environment finding in Section 4.6 is the current-best evidence, and itself already shows a comparable qualitative pattern (DQN’s reward advantage reaching significance while BMPP-DQN remains strongest on the independent multi-criteria check) without needing the larger sample to surface it this time.

4.13 Discussion: Answering the Research Questions

RQ1 (architectural integration, Section 1.3): BMPP-DQN trains stably end-to-end as a single coupled network combining branching discrete decomposition with multi-pass continuous parameter processing (Chapter 3), across all 3 seeds, with no divergence or training-instability failures observed, under the corrected environment and algorithm (Section 4.2). This is an architectural and training-stability claim, evidenced by the convergence behavior in Section 4.3 and the consistent final performance across seeds in Table 4.1; it does not by itself establish that the architecture integrates its two halves *efficiently*, which Section 4.7’s finding (a simple heuristic beats BMPP-DQN’s reward) weighs against.

RQ2 (energy-saving performance): the evidence does not support a single-metric energy-saving win, and does so more starkly than the pre-fix chapter found. BMPP-DQN’s reward is now significantly *worse* than DQN’s ($p = 0.015$), and both a trivial heuristic and a privileged oracle significantly beat every trained method’s reward, BMPP-DQN included (Section 4.7). Objective 4’s $\geq 15\%$ energy-saving target against the trained baselines is, without qualification, **not met** by any result in this chapter – and the calibration check now shows the gap is not merely “no baseline beats BMPP-DQN by 15%,” but “an untrained rule beats every trained method including BMPP-DQN at all.” What the evidence *does* support is a genuine multi-criteria balance: BMPP-DQN ranks first or second under every MCDA scheme tested among the four trained methods (Sections 4.5–4.6), the only one of the four with that consistency, even though it is not the best-reward method. Section 4.8’s fair-cadence control qualifies part of this picture: BMPP-DQN’s switching-frequency advantage

over DQN/MP-DQN at their own default cadence is substantially a cadence artifact, not an architectural one – at matched cadence, both baselines switch *less* than BMPP-DQN. The multi-criteria-balance finding (Sections 4.5–4.6) does not rest on switching frequency alone, but switching frequency is one of its five inputs, and this chapter does not claim architectural credit for an advantage now shown to be substantially cadence-driven.

RQ3 (two-timescale design): directly answered in Section 4.10. Collapsing the two-timescale separation to a single timescale reduces switching stability by $3.6\times$ but *improves* reward and throughput at the cost of higher power – a genuine trade-off, not the previously-reported free reduction in switching. The two-timescale design’s contribution is decision stability at the discrete-action level, purchased at a measurable reward/power cost, matched to the Non-RT/Near-RT RIC timescale separation this problem is built on (Section 1.1).

Chapter 5 returns to these three answers, including the honest reconciliation Section 4.7 and Section 4.8 now require, alongside this thesis’s scope and limitations (Section 1.6) to state its overall contribution and directions for future work.

Chapter 5

Conclusion and Future Work

5.1 Summary of Contributions

This thesis designed, implemented, and evaluated BMPP-DQN (Branching Multi-Pass Parameterized DQN), a deep reinforcement learning architecture combining branching discrete-action decomposition [5] with multi-pass parameterized continuous-action processing [3, 4] within a single coupled network, for joint RU activation, functional split selection, transmit power control, and PRB allocation in a simulated O-RAN energy-optimization scenario. Four contributions, each evidenced in Chapters 3–4, follow from this design:

1. **An architectural integration not previously proposed** (Section 1.5): BMPP-DQN couples branching’s R -independent discrete decomposition with multi-pass processing’s principled discrete-continuous parameter coupling, closing the gap identified in Section 1.2 between OREO’s flat, thresholded continuous relaxation [1] and the two architectures this design draws on, neither of which alone handles O-RAN’s hybrid, multi-timescale action space.
2. **A formal parameterized MDP formulation** (Section 3.5) spanning four action branches across two decision timescales, with a fronthaul- and switching-aware reward function (Eq. 3.21) and a literature-informed power model (Section 3.4).
3. **A four-method comparison under matched conditions** (Chapter 4) against DQN, DDPG, and MP-DQN, cross-checked by three independent multi-criteria decision-analysis schemes (equal-weighted TOPSIS, entropy-weighted TOPSIS, and VIKOR; Sections 4.5–4.6) – a more extensive robustness-checking protocol than this thesis’s own original scope required, undertaken specifically to avoid over-claiming from a single table of $n = 3$ results. A power-model sensitivity sweep and a supplementary $n = 10$ -seed validation were also run (Sections 4.9, 4.12), though both predate, and have not been repeated against, the bug fixes in the next item.
4. **Three environment/algorithm bugs found and fixed via independent review** (Section 4.2): a capacity- inflation error in the PRB-sharing formula, a diurnal clock advancing $36,000\times$ too fast, and a missing γ^N discount in BMPP-DQN’s own upper-level TD target. All three affected every number this chapter and Chapter 4 report;

finding and fixing them, with regression tests and a full retrain, is itself a methodological contribution of this thesis, not merely a correction.

5. **A heuristic/oracle calibration check and a fair-cadence control** (Sections 4.7–4.8), neither originally planned, added specifically to test whether the four-method comparison’s own findings held up against simpler reference policies and an unfounded decision cadence. Both materially changed this thesis’s conclusions, reported in Section 4.13 below.
6. **A direct empirical answer to RQ3** (Section 4.10): the two-timescale design’s measurable contribution is discrete-decision stability, at a measurable cost in reward and power rather than for free, resolving a research question this track’s concept note had posed but left empirically untested until this thesis’s own ablation.

5.2 Revisiting the Research Objectives

Section 1.4 stated five research objectives. Objectives 1-3 (design BMPP-DQN; formulate the parameterized MDP; implement the focused single-gNB environment) are met in full, as documented in Chapter 3. Objective 5 (analyze the two-timescale design’s effect on convergence and energy efficiency) is answered directly in Section 4.10: the design improves decision stability without a measurable convergence or efficiency cost in either direction.

Objective 4 – quantify BMPP-DQN’s energy-saving performance against DQN, DDPG, and MP-DQN, against a target of $\geq 15\%$ savings relative to these baselines – is stated here, as Section 1.4 already flagged it would be, without qualification: **this target is not met by the evidence gathered**, under the corrected environment (Section 4.2) any more than it was before. BMPP-DQN’s mean power (142.6 W) is now the *lowest* of the four methods – narrowly below MP-DQN’s 150.2 W, a margin well short of 15% – but its reward is significantly *worse* than DQN’s ($p = 0.015$), and Section 4.7’s calibration check adds a more consequential finding than either: a simple, untrained heuristic and a privileged, non-causal oracle both significantly outperform BMPP-DQN’s reward, and every other trained method’s. This objective is restated here exactly as it was originally planned and investigated, rather than retroactively reworded to match the evidence gathered after the fact, per this track’s own stated discipline (Section 1.4’s own forward-reference to this chapter).

What the evidence *does* support, in place of a raw energy-savings margin, is detailed in Section 4.13: among the four trained methods, BMPP-DQN ranks first or second (never worse) under every multi-criteria scheme tested (Sections 4.5–4.6), the only one of the four with that consistency – a cleaner version of this finding than the pre-fix chapter reported, since the best-reward method (DQN) is simultaneously the *least* consistent multi-criteria performer. Section 4.8’s fair-cadence control qualifies this finding, however: BMPP-DQN’s switching-frequency advantage over DQN/MP-DQN at their own default cadence, one of the five inputs to this multi-criteria check, is substantially a decision-cadence artifact rather than an architectural one – at matched cadence, both baselines switch less than BMPP-DQN itself. The multi-criteria-balance finding is a genuine, evidenced contribution, but this chapter does not claim architectural credit for an advantage now shown to be substantially cadence-driven,

and it is a different claim in kind from a demonstrated energy-savings percentage; the two are not conflated here or in Section 5.3 below.

5.3 Significance of the Research, Revisited

Section 1.5 stated this thesis’s intended significance before its results were known. Revisited in light of Chapter 4:

- **Architectural contribution** (DRL literature on hybrid action spaces): unaffected by the results – the integration of branching and multi-pass processing into a single coupled network is a design contribution independent of how it ultimately performs, and Objective 1/RQ1 (Section 4.13) confirm it trains stably across every seed tested.
- **O-RAN application relevance**: unaffected – the problem formulation (Chapter 3) remains a faithful structural match to O-RAN’s disaggregated RU/DU/CU control surface and its Non-RT/Near-RT RIC timescale split, independent of this specific algorithm’s comparative performance.
- **Energy savings**: *not demonstrated* (Section 5.2). This bullet is retained, not deleted, because it was this thesis’s original stated significance claim and the evidence against it is itself part of this thesis’s record, not something to omit – and the corrected environment makes this conclusion more decisive, not less: a trivial heuristic now beats every trained method’s reward, BMPP-DQN included (Section 4.7).
- **Multi-criteria balance** (evidenced, among the four trained methods): BMPP-DQN is the most consistently-ranked of the four methods across power, strict QoS, per-UE QoS, switching stability, and throughput simultaneously, first or second under all three independent MCDA schemes tested at $n = 3$ (Section 4.6). This is this thesis’s actual, defensible empirical contribution relative to RQ2, with two qualifications carried forward honestly rather than smoothed over: part of its switching-frequency input is a cadence artifact, not an architectural one (Section 4.8), and whether this ranking holds at $n = 10$ or under power-model perturbation is unknown under the corrected environment – both checks predate the bug fixes (Sections 4.9, 4.12) and were not rerun against them in this revision.

5.4 Limitations

Section 1.6 bounded this thesis’s scope in advance; those same bounds are this chapter’s limitations, read in light of what was actually found:

- **Simulation-only evaluation.** All results use a simplified log-distance-path-loss-plus-Rayleigh-fading channel model (Section 3.2) and a literature-informed but not independently validated power-consumption model (Section 3.4.1). No real O-RAN testbed, hardware power measurement, or over-the-air channel was used. The power-model sensitivity sweep (Section 4.9) shows the *comparative* findings are insensitive

to this model’s unvalidated constants’ order of magnitude, but this does not establish that the *absolute* power, reward, or QoS figures reported in this chapter would transfer to a physical deployment.

- **Single-gNB, downlink-only scope.** The environment models one gNB with $R = 4$ RUs, one DU, and one CU (Section 3.1); no multi-cell coordination, inter-cell interference beyond the single-pool model already included, or uplink traffic is considered.
- **A tractable, 3-option split abstraction.** The three centralization levels used here approximate 3GPP TR 38.801 Options 2, 6, and 8 (Section 3.1) rather than the full space of standardized functional splits.
- **Unvalidated reward weights.** The reward function’s three weights (α , β_{qos} , γ_{sw} , Section 3.5) were chosen to preserve a reasonable relative magnitude, not derived from a literature source or tested in a sensitivity sweep of their own – unlike the power-model constants, which were swept in Section 4.9. Given that BMPP-DQN’s per-UE QoS deficit relative to the baselines was investigated across several training-mechanics explanations without this one being tested (the independent-exploration fix, PER, and curriculum learning were all ruled in or out; reward-weight sensitivity was not), this remains the most likely untested lever on the energy/QoS/switching trade-off reported in Chapter 4.
- **Thin statistical power at the canonical scale, and an $n = 10$ check that predates the bug fixes.** The supervisor-approved protocol’s $n = 3$ seeds is a thin basis for the paired significance tests reported throughout Chapter 4. A supplementary $n = 10$ validation (Section 4.12) was run before Section 4.2’s fixes and changed which comparisons reached significance under the pre-fix environment; it has not been rerun against the corrected code, so whether the same sample-size sensitivity holds now is open.
- **Trained methods underperforming a simple heuristic.** Section 4.7 found that an untrained, fixed-rule heuristic beats every trained method’s reward, BMPP-DQN included, at this thesis’s training budget (500 episodes, 3 seeds). This is a limitation of what this thesis’s experiments establish about training effectiveness at this budget, not only of the environment or reward design, and is not resolved by anything in this thesis.
- **Three baselines, not an exhaustive comparison.** BMPP-DQN is compared against DQN, DDPG, and MP-DQN (Section 1.6), plus the heuristic/oracle calibration pair added in this revision (Section 4.7); OREO [1] is discussed only qualitatively (Section 1.2), and HyAR [32], a VAE-based hybrid-action alternative considered but not adopted for this architecture, is likewise not reproduced as a baseline.

5.5 Future Work

Five directions follow directly from the limitations above and from findings in Chapter 4 that this thesis’s own scope could investigate but not resolve.

Rerun the sensitivity sweep and $n = 10$ validation against the corrected environment. Section 4.2’s fixes change every method’s absolute metrics substantially; whether the pre-fix power-model-insensitivity finding (Section 4.9) and the pre-fix $n = 10$ ranking-volatility finding (Section 4.12) still hold is unknown and was explicitly out of scope for this revision.

Investigate why trained methods underperform a simple heuristic. Section 4.7’s calibration check is this thesis’s most consequential open question: at this training budget (500 episodes, 3 seeds), none of the four trained methods, BMPP-DQN included, beat an untrained fixed-rule heuristic’s reward. Three candidate explanations have since been tested, all for DQN specifically. Two were ruled out: insufficient training (Section 4.7.1 retrained at $4\times$ the episode budget; gap to the heuristic moved from 32.0% to only 30.8% of DQN’s own reward magnitude) and un-normalized reward scale feeding the Bellman target (Section 4.7.2 scaled it by 0.01; the gap moved to 30.7%, an equally marginal change). The third, DQN’s exploration schedule, was not: slowing ϵ -decay so $\approx 4.5\times$ more residual exploration remained at episode 500 (Section 4.7.3) narrowed the gap to 27.3%, more than $3\times$ either other check’s effect – a real, if partial, contribution. Exploration is therefore the leading confirmed lever of the three, though over a quarter of the deficit remains even with substantially more exploration, so it is not the whole explanation either. The remaining candidate explanations this thesis did not test are compounding the exploration-schedule change further (an even slower decay, or exploration persisting past episode 500), hyperparameters tuned for the pre-fix (capacity-inflated) environment never being retuned for the corrected one, and the unvalidated reward-weight *magnitudes* themselves (the limitation above) – distinct from the reward-*scale* question already ruled out, since reweighting the energy/QoS/switching terms relative to each other changes what an optimal policy even is, where rescaling the whole reward uniformly (as tested) does not.

Validate the power model and reward weights against real measurements. Section 3.4.1’s needs-validation constants (the absolute RU/DU/CU/fronthaul wattage figures) and Section 3.5’s reward weights are the two remaining unvalidated numerical components of this thesis’s model. The pre-fix power-model sensitivity sweep (Section 4.9) suggested the comparative ranking was insensitive to the former’s order of magnitude, but this has not been re-established under the corrected environment (the future-work item above); whether the same holds for the reward weights, and whether either can be replaced with commercial O-RAN hardware measurements, is open regardless. A real-testbed study in the style of Bordin et al.’s ns-O-RAN-based evaluation [6] would address both simultaneously.

Extend beyond a single gNB. This thesis’s single-agent, single-gNB scope was a deliberate tractability bound (Section 1.6), justified in part by Liang et al.’s federated-TD3 O-RAN energy-optimization work [8] showing a feasible path to multi-cell coordination via an rApp aggregator plus per-cell xApp agents, rather than a single centralized policy. Extending BMPP-DQN’s branching-plus-multi-pass design to that federated setting – one BMPP-DQN instance per cell, aggregated rather than jointly trained – is a natural next step that preserves this thesis’s core architectural contribution while removing its single-gNB limitation.

Test functional-split abstractions beyond the 3-option mapping. Section 3.1’s three-level split abstraction is a tractability simplification whose bandwidth ratios do not match 3GPP TR 38.801’s real per-option figures exactly (Section 3.4.1’s own disclosure). A follow-up could use the full 8-option split space directly, at the cost of a larger discrete action

space than this thesis’s branching architecture was sized for, testing whether BMPP-DQN’s branching decomposition scales to that larger space without the flat baselines’ (MP-DQN’s, in particular) combinatorial tractability ceiling.

5.6 Closing Statement

This thesis set out to answer whether branching action decomposition and multi-pass parameterized processing could be combined into a single DRL architecture for O-RAN energy optimization, and whether doing so would yield energy savings over existing discrete, continuous, and parameterized baselines. The first question is answered affirmatively: BMPP-DQN trains stably, end-to-end, as the proposed coupled architecture (Section 5.1). The second is answered in the negative, more starkly than this thesis originally anticipated: under an environment corrected of three independently-identified bugs (Section 4.2), not only does no trained baseline’s power beat BMPP-DQN’s by 15%, a simple untrained heuristic and a privileged per-step oracle both beat every trained method’s reward outright (Section 4.7). Stated plainly rather than reframed after the fact (Section 5.2), this is a more consequential negative result than a missed percentage target. What the evidence gathered in its place supports – a multi-criteria balance across power, QoS, and switching stability among the four trained methods specifically, with its switching-frequency component now shown to be partly cadence-driven rather than purely architectural (Section 4.8) – is a narrower and more qualified contribution than originally targeted, but one this thesis can defend with the evidence actually in hand, which is the standard Chapter 4 and this chapter have tried to hold themselves to throughout, including when that evidence turned out less favorably than earlier drafts of this same chapter had reported.

References

- [1] M. M. H. Qazzaz, A. Salama, M. Hafeez, and S. A. R. Zaidi, “OREO: Open RAN Energy Optimization via Deep Reinforcement Learning for 6G Networks,” *IEEE Open Journal of the Communications Society*, vol. 7, pp. 4165–4182, 2026.
- [2] O-RAN Alliance, “O-RAN Minimum Viable Plan and Commercialization Strategy,” Jun. 2021.
- [3] J. Xiong, Q. Wang, Z. Yang, P. Sun, L. Han, Y. Zheng, H. Fu, T. Zhang, J. Liu, and H. Liu, “Parametrized Deep Q-Networks Learning: Reinforcement Learning with Discrete-Continuous Hybrid Action Space,” arXiv preprint arXiv:1810.06394, 2018.
- [4] C. J. Bester, S. D. James, and G. D. Konidakis, “Multi-Pass Q-Networks for Deep Reinforcement Learning with Parameterised Action Spaces,” arXiv preprint arXiv:1905.04388, 2019.
- [5] A. Tavakoli, F. Pardo, and P. Kormushev, “Action Branching Architectures for Deep Reinforcement Learning,” in *Proc. AAAI Conference on Artificial Intelligence*, 2018.
- [6] M. Bordin, A. Lacava, M. Polese, S. Satish, M. AnanthaSwamy Nittoor, R. Sivaraj, F. Cuomo, and T. Melodia, “Design and Evaluation of Deep Reinforcement Learning for Energy Saving in Open RAN,” in *Proc. IEEE 22nd Consumer Communications & Networking Conference (CCNC)*, Las Vegas, NV, USA, Jan. 2025, pp. 1–6.
- [7] A. Sthankiya et al., “A Survey on AI-Driven Energy Optimization in Next-Generation RAN,” arXiv preprint arXiv:2411.02164, 2024.
- [8] X. Liang, A. Al-Tahmeesschi, S. B. Chetty, and H. Ahmadi, “Green O-RAN Operation: a Modern ML-Driven Network Energy Consumption Optimization,” in *Proc. IEEE GLOBECOM*, 2025, pp. 4475–4480.
- [9] M. Sohaib et al., “DRL Transfer Learning for Cloud-Native O-RAN,” arXiv preprint arXiv:2407.11563, 2024.
- [10] A. Iqbal, M.-L. Tham, and Y. C. Chang, “Double Deep Q-Network-Based Energy-Efficient Resource Allocation in Cloud Radio Access Network,” *IEEE Access*, vol. 9, pp. 20440–[end page not independently confirmed in this repository], 2021, doi: 10.1109/ACCESS.2021.3054909.

- [11] M. Fathy, M. S. Abood, and M. M. Hamdi, "Optimization of Energy-Efficient Cloud Radio Access Networks for 5G using Neural Networks," in *Proc. Int. Conf. Intelligent Technology, System and Service for Internet of Everything (ITSS-IOE)*, 2021.
- [12] W. H. A. Al-Zubaedi, "Planning a C-RAN Deployment for the Next Generation Cellular Networks," Ph.D. dissertation, Dept. of Electronic and Computer Engineering, Brunel University London, London, U.K., 2019.
- [13] H. Shengren, E. M. Salazar Duque, P. P. Vergara, and P. Palensky, "Performance Comparison of Deep RL Algorithms for Energy Systems Optimal Scheduling," in *Proc. IEEE PES Innovative Smart Grid Technologies Conference Europe (ISGT-Europe)*, 2022, pp. 1–6.
- [14] "Self-Optimized Agent for Load Balancing and Energy Efficiency: A Reinforcement Learning Framework With Hybrid Action Space," *IEEE Trans. on Network and Service Management*, vol. 21, no. 4, pp. 4902–4919, Jul. 2024.
- [15] S. Sharma and W. Yoon, "Energy Efficient Power Allocation in Massive MIMO Based on Parameterized Deep DQN," *Electronics*, vol. 12, no. 21, p. 4517, 2023.
- [16] H. Kabir, M.-L. Tham, and Y. C. Chang, "Mobility-Aware Resource Allocation in IoT Network for Post-Disaster Communications with Parameterized Reinforcement Learning," *Sensors*, vol. 23, no. 14, p. 6448, Jul. 2023.
- [17] J. Lu, P. Yan, and H. Zeng, "EEExApp: GNN-Based Reinforcement Learning for Radio Unit Energy Optimization in 5G O-RAN," in *Proc. IEEE INFOCOM*, 2026, pp. 1–10.
- [18] C. Yan et al., "Hybrid Reinforcement Learning in Parameterized Action Space via Fluctuates Constraint," *Engineering Applications of Artificial Intelligence*, vol. 162, p. 112499, 2025.
- [19] F. Rezazadeh, H. Chergui, L. Christofi, and C. Verikoukis, "Actor-Critic- Based Learning for Zero-Touch Joint Resource and Energy Control in Network Slicing," arXiv preprint, 2022.
- [20] E. Amiri, N. Wang, M. Shojafar, and R. Tafazolli, "Energy-Aware Dynamic VNF Splitting in O-RAN Using Deep Reinforcement Learning," in *Proc. IEEE Int. Conf. Local Computer Networks (LCN)*, 2022, pp. 422–429.
- [21] R. Barker, T. Seyfi, and F. Afghah, "Advancements in Mobile Edge Computing and Open RAN: Leveraging Artificial Intelligence and Machine Learning for Wireless Systems," *arXivLens*, 2025.
- [22] N. Lassoued and N. Boujnah, "A Comprehensive Review of Energy Efficiency in 5G Networks: Past Strategies, Present Advances, and Future Research Directions," *Computers*, vol. 15, no. 1, p. 50, 2026.

- [23] Chuang et al., “[Title and venue not independently verified in this repository – known only via a descriptive secondary reference, per AGENTS.md’s Foundational References table:] Hybrid A3C and Dueling DQN applied to industrial IoT task scheduling in a 5G Cloud-RAN setting,” 2025. *Needs primary-source verification before this entry is used outside this draft.*
- [24] 3GPP, “Study on New Radio Access Technology: Radio Access Architecture and Interfaces,” TR 38.801, V0.4.0, Aug. 2016.
- [25] 3GPP, “Study on Network Energy Savings for NR,” TR 38.864, 2024.
- [26] G. Auer et al., “How Much Energy Is Needed to Run a Wireless Network?,” *IEEE Wireless Communications*, vol. 18, no. 5, pp. 40–49, Oct. 2011.
- [27] Benetel, “RAN550 Outdoor/Indoor O-RU Datasheet (n78/n79, Split 7.2x),” product datasheet. Max. TX output power (total EIRP): 2 W (33 dBm).
- [28] C.-L. Hwang and K. Yoon, *Multiple Attribute Decision Making: Methods and Applications*. Berlin, Germany: Springer-Verlag, 1981.
- [29] M. Zeleny, *Multiple Criteria Decision Making*. New York, NY, USA: McGraw-Hill, 1982.
- [30] S. Opricovic and G.-H. Tzeng, “Compromise solution by MCDM methods: A comparative analysis of VIKOR and TOPSIS,” *European Journal of Operational Research*, vol. 156, no. 2, pp. 445–455, 2004.
- [31] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Hillsdale, NJ, USA: Lawrence Erlbaum Associates, 1988.
- [32] Y. Li, Q. Zhang, Y. Wang, H. Jianye, and Y. Yu, “HyAR: Addressing Discrete-Continuous Action Space via Hybrid Action Representation,” in *Proc. International Conference on Learning Representations (ICLR)*, 2022.

Appendix A

Supplementary Material

A.1 Default Configuration Listing

The full contents of `config/oran_default.yaml`, the single configuration file governing every result in Chapter 4 (canonical $n = 3$ run) unless stated otherwise (the $n = 10$ validation and the RQ3/sensitivity ablations each change only the one parameter named where they are introduced). Every constant’s needs-validation status is disclosed inline, in this same file, at the point each constant is defined – not repeated separately here.

```
# Default Configuration for O-RAN / BMPP-DQN Experiments
# ORAN_BMPP_DQN_Concept_Note_v1.md -- a separate, additive research track
# alongside config/default.yaml’s C-RAN track. No keys/sections here are
# read by cran_env/agents/training/evaluation, and vice versa.

# =====
# Network Topology (Concept Note Section 10.3: single-gNB, focused scope;
# 2026-08-30 literature check of DQRL/OREO’s own scenario scales -- see
# Section 10.3’s own note -- shows this UE:RU ratio sits within
# precedent ranges, though the absolute RU count is still smaller than
# either paper’s own scenario)
# =====
network:
  n_ru: 4
  n_du: 1
  n_cu: 1
  n_ue: 8
  n_splits: 3          # 3GPP TR 38.801 Options 2/6/8, mapped to centralization
                        # levels c=0/1/2 -- Concept Note Section 10.2 (needs validation)

  area_size_m: 500
  carrier_freq_ghz: 3.5
  # 2026-09-24: raised from 20 -> 100 MHz. Empirical check (best-case
  # all-RU/max-power/equal-PRB policy, 300 steps): at 20 MHz, mean
  # aggregate demand (~101 Mbps, driven by this scenario’s Poisson
  # burstiness) already approached mean achievable throughput (~77 Mbps)
  # even under that generous reference policy, leaving essentially no
  # headroom for any policy -- including the intended learned ones -- to
  # satisfy QoS. 100 MHz raises mean achievable throughput to ~244 Mbps
  # under the same reference policy (best-case per-UE satisfaction
```

```

# 74%→81%), giving real headroom while still being a standard,
# predated channel bandwidth for this model's own n78-band carrier
# frequency (3.5 GHz) -- not an arbitrary tuning choice. See
# docs/oran_thesis_guide.md's QoS-metric note for the accompanying
# per-UE-vs-all-UE metric distinction this same investigation surfaced.
# 2026-10-01 re-check after fixing oran_env.py's PRB-sharing bug (each
# co-served UE was previously scored against its serving RU's *full*
# share, not divided among every UE that RU simultaneously served --
# see Chapter 4's bug-fix disclosure): the ~384 Mbps/84% figures above
# were computed under that inflated formula and are superseded by the
# ~244 Mbps/81% figures now stated; 100 MHz still clears this headroom
# check under the corrected formula (~2.4x margin), so the constant
# itself did not need to change, only this comment's own numbers.
bandwidth_mhz: 100
noise_power_dbm: -102

# =====
# Channel Model (simplified: log-distance path loss + Rayleigh fading only,
# no shadowing/temporal correlation -- Concept Note Section 5.1)
# =====
channel:
    path_loss_exponent: 3.5

# =====
# Traffic Model (trapezoidal daily envelope x Poisson per-UE arrivals,
# Concept Note Section 5.1; needs-validation placeholders per Section 10.6.
# 2026-08-30: lambda_peak/packet_size_bits below are now derived directly
# from 3GPP TR 38.864 Annex A's FTP Model 3 (0.5 MB packet size, 200 ms
# mean inter-arrival time -- a real, standard 3GPP Poisson traffic model),
# not a guess -- see Section 10.6's own note. floor_ratio and t1-t4 remain
# unvalidated placeholders: TR 38.864's own load scenarios (Table A-1) are
# instantaneous PRB-utilization snapshots, not tied to time-of-day, and its
# own study scope stops at "medium load" (<=50%), so it gives no floor:peak
# ratio or diurnal timing to derive these from.
# =====
traffic:
    # 3GPP TR 38.864 Annex A FTP Model 3: 200ms mean inter-arrival = 5
    # arrivals/s = 0.5 per 0.1s step (peak Poisson rate, per UE)
    lambda_peak: 0.5
    floor_ratio: 0.2                # off-peak floor as a fraction of lambda_peak
                                    # -- still an unvalidated placeholder (see note above)
    # 3GPP TR 38.864 Annex A FTP Model 3: 0.5 MB packet/file size = 4e6 bits
    packet_size_bits: 4.0e+6
    step_duration_s: 0.1            # matches the 10-100ms Near-RT RIC decision rate
    t1: 7.0                         # rise start (hour of day) -- still unvalidated
    t2: 10.0                        # plateau start -- still unvalidated
    t3: 20.0                        # plateau end -- still unvalidated
    t4: 23.0                        # fall end -- still unvalidated

# =====
# Power Model (RU/DU/CU/fronthaul, monotonic in split centralization level;
# Concept Note Section 10.5). Most constants below remain needs-validation
# literature-style placeholders; p_max_dbm (below) is the one exception,
# now derived from a real small-cell O-RU datasheet -- see its own comment.

```

```

# =====
power:
  ru:
    p_sleep_w: 2.0
    p_switch_w: 2.0          # RU activation-flip switching cost
    p_switch_split_w: 1.0    # RU split-change switching cost
    pa_efficiency: 0.25
    # 2026-08-30: 33 dBm = 2 W, matching the Benetel RAN550's real max TX
    # output power (total EIRP) for a Split-7.2x indoor small-cell O-RU
    # (was 30 dBm/1 W, an unvalidated placeholder) -- see Concept Note
    # Section 10.5's own note. Not a guess: same physical quantity, same
    # units, a real commercial small-cell O-RU datasheet.
    p_max_dbm: 33
    # Active-RU processing power per split centralization level c=0/1/2
    # (Concept Note Section 10.2), c=0 (Option 2) = most RU-side processing
    # down to c=2 (Option 8) = least. Previously not exposed here at all --
    # oran_env/oran_env.py never passed this array to ORANPowerModel, so
    # only its Python-side default (unchanged below) was ever actually
    # used; now genuinely config-driven, matching every other constructor
    # argument's existing convention.
    p_proc_by_split_w: [10.0, 6.0, 3.0]

  du:
    p_static_w: 50.0
    # DU processing power contributed per active RU at split level c
    # (increasing with c, since less-centralized splits push more
    # processing to the DU) -- same previously-not-wired gap as
    # ru.p_proc_by_split_w above.
    p_per_ru_by_split_w: [5.0, 10.0, 20.0]

  cu:
    p_static_w: 30.0
    p_dyn_per_ru_w: 1.0

  fronthaul:
    p_common_w: 10.0
    # Fronthaul power contributed per active RU at split level c
    # (increasing with c, since less-centralized splits need more
    # fronthaul bandwidth) -- same previously-not-wired gap as above.
    p_per_ru_by_split_w: [3.0, 6.0, 15.0]

# =====
# DRL Algorithm (BMPP-DQN: two-timescale upper/lower Q-networks, no TD3 --
# Concept Note Section 5.2/10.4)
# =====
algorithm:
  name: "bmpp_dqn"
  # 2026-09-29: reverted back to 500 (was briefly raised to 1000 on
  # 2026-09-26; see docs/daily_log.md's 2026-09-26 entry for that
  # rationale, still valid as a finding, not retracted) -- reverted at the
  # user's explicit request, back to this thesis's originally-reported
  # 3-seed/500-episode scale (Concept Note Section 6.5). The 5-seed/
  # 1000-episode expanded results (Section 6.6-6.7) remain archived at
  # data/results_oran_archive/5seed_1000ep_pre_exploration_fix_20260929/,

```

```

# not discarded.
max_episodes: 500
max_steps_per_episode: 100

# Two-timescale cadence: discrete (upper) decisions held for N lower-level
# steps -- Concept Note Section 5.2 ("1-10s" upper vs "10-100ms" lower;
# at step_duration_s=0.1, N=10 is the low end of that range, needs validation)
upper_level_period_steps: 10

# Replay buffers: separate lengths per Concept Note Section 5.2
# ("longer replay buffers" for upper-level, "faster updates" for lower-level)
upper_buffer_size: 200000
lower_buffer_size: 1000000
batch_size: 128
min_buffer_size: 2000

lr_discrete: 1.0e-4
lr_actor: 3.0e-4
lr_critic: 3.0e-4          # Used by the DDPG baseline (oran_agents/ddpg_agent.py)

# Smaller than the C-RAN track's [256, 128] -- matches this track's
# smaller single-gNB scope (Concept Note Section 5.1)
hidden_dims: [128, 64]
activation: "relu"
use_layer_norm: true

gamma: 0.99
tau: 0.005

epsilon_start: 1.0
epsilon_end: 0.01
epsilon_decay: 0.995

continuous_noise_std: 0.1
continuous_noise_std_end: 0.01

gradient_clip_norm: 1.0

# Tractability cap for the flat MP-DQN baseline's joint action space
# ( $2^{n_{ru}} * n_{splits}^{n_{ru}}$ ); mirrors config/default.yaml's analogous cap
# for its own MP-DQN baseline.
max_n_ru_for_flat_joint_action: 6

# =====
# Reward Function Weights (needs validation -- see Concept Note
# Section 10.7; unlike every power/traffic/split constant in this file,
# these three weights have never been subjected to a literature check or
# a sensitivity analysis, despite directly defining the optimization
# objective every result in this thesis is measured against.
# oran_env/oran_env.py's step(): reward = alpha_energy * (throughput_mbps
# / total_power_w) - beta_qos * (qos_shortfall_mbps) - gamma_switch *
# (switching_events). Chosen only to preserve a reasonable relative
# magnitude -- beta_qos=10 so a QoS violation (Mbps of shortfall)
# dominates the throughput-per-watt term, gamma_switch=0.5 as a mild,

```

```

# not dominant, switching penalty -- not derived from any source.
# 2026-09 sensitivity sweep (Concept Note Section 6.9) perturbs the
# power-model constants these weights are combined with, not the
# weights themselves; the weights remain open for future work.
# =====
reward:
  alpha_energy: 1.0
  beta_qos: 10.0
  gamma_switch: 0.5

# =====
# Evaluation (Concept Note Section 5.3: 3 seeds, per RQ2)
# =====
evaluation:
  eval_freq: 50
  n_eval_episodes: 5
  n_random_seeds: 3   # [42, 123, 456] -- Concept Note Section 5.3 "3 random seeds"
  save_checkpoints: true
  checkpoint_freq: 250

# =====
# Hardware
# =====
hardware:
  device: "cuda"   # cuda or cpu (falls back to cpu automatically if no GPU
                  # is available; oran_agents/*.py read this as each
                  # agent's device default)

```

A.2 Per-Seed Raw Results: Canonical Comparison ($n = 3$)

Table A.1 gives the individual per-seed final evaluation metrics each method’s mean row in Table 4.1 (Section 4.4) aggregates, read directly from each run’s own `summary.json` (`data/results_oran/`). “QoS Rate” and “Per-UE QoS” follow the same two definitions given in Section 4.1.

DDPG’s identical 0.040 switching frequency across all three seeds is a clean architectural fact, not an artifact of any one seed: unlike an earlier version of this run (made under an environment-physics bug since fixed, see Chapter 4’s bug-fix disclosure), no seed here shows a collapsed, nonsensically-low throughput alongside DDPG’s uniform switching count – all three seeds’ throughput (197-249 Mbps) and power (204-207 W) are mutually consistent. The 0.040 figure is simply what Section 4.4 already attributes it to: DDPG’s purely continuous relaxation architecturally never flips a discrete decision in the sense `switching_events` counts. MP-DQN’s seed 456 shows the same 0.040 value by coincidence of its own learned policy settling on a stable split/activation choice for that seed specifically, not for the same architectural reason – MP-DQN can and does switch (seeds 42/123 show 0.468/0.354).

Table A.1: Per-seed final evaluation metrics, canonical $n = 3$ run, all 4 methods.

Algorithm	Seed	Reward	Power (W)	QoS Rate (%)	Per-UE QoS (%)	Switching	Throughput (Mbps)
BMPP-DQN	42	-69424.60	151.49	22.8	78.43	0.236	230.37
BMPP-DQN	123	-67526.27	145.08	22.0	77.72	0.184	194.92
BMPP-DQN	456	-72152.45	131.35	22.2	77.53	0.124	178.55
DDPG	42	-87914.28	207.14	21.8	76.78	0.040	247.24
DDPG	123	-100234.78	206.07	21.0	74.23	0.040	197.37
DDPG	456	-88119.47	204.20	20.4	76.70	0.040	249.02
DQN	42	-56464.21	234.47	23.6	79.00	0.654	260.76
DQN	123	-58656.24	196.20	22.4	79.00	1.128	251.89
DQN	456	-58649.02	203.00	22.4	79.00	0.856	246.02
MP-DQN	42	-101610.51	142.30	20.0	73.00	0.468	52.33
MP-DQN	123	-77158.25	158.76	22.2	75.33	0.354	122.69
MP-DQN	456	-67004.92	149.41	28.4	78.83	0.040	260.51

A.3 Per-Seed Raw Results: RQ3 Two-Timescale Ablation

Table A.2 gives the per-seed figures underlying Section 4.10’s RQ3 ablation, for the same three seeds under both configurations: the two-timescale default (`upper_level_period_steps=10`, identical to the BMPP-DQN rows of Table A.1) and the single-timescale ablation (`upper_level_period_steps=1`, `config/rq3.single.timescale.yaml`).

Table A.2: Per-seed final evaluation metrics, RQ3 ablation: two-timescale (proposed, $N = 10$) vs. single-timescale ($N = 1$) BMPP-DQN.

Configuration	Seed	Reward	Power (W)	QoS Rate (%)	Per-UE QoS (%)	Switching	Throughput (Mbps)
Two-Timescale ($N=10$)	42	-69424.60	151.49	22.8	78.43	0.236	230.37
Two-Timescale ($N=10$)	123	-67526.27	145.08	22.0	77.72	0.184	194.92
Two-Timescale ($N=10$)	456	-72152.45	131.35	22.2	77.53	0.124	178.55
Single-Timescale ($N=1$)	42	-64889.10	233.19	23.8	79.00	0.566	275.84
Single-Timescale ($N=1$)	123	-61302.61	239.14	23.2	77.68	0.846	233.94
Single-Timescale ($N=1$)	456	-59417.14	232.37	20.6	78.65	0.520	231.03

Every single-timescale seed’s switching frequency exceeds every two-timescale seed’s, with no overlap (minimum single-timescale value 0.520 vs. maximum two-timescale value 0.236) – the clearest seed-level evidence for Section 4.10’s conclusion that the two-timescale design’s effect on switching stability is consistent across seeds individually, not only in the three-seed mean. Unlike the earlier, pre-bug-fix version of this ablation, the two configurations here are *not* reward/power/QoS-neutral: every single-timescale seed also shows higher power and (mostly) higher reward than every two-timescale seed, a genuine trade-off discussed in Section 4.10, not a free reduction in switching.

A.4 Per-Seed Raw Results: Fair-Cadence Control

Table A.3 gives the per-seed figures underlying Section 4.8’s fair-cadence control: DQN and MP-DQN retrained and re-evaluated with the same `upper_level_period_steps=10` discrete-decision hold BMPP-DQN uses by default (`discrete_hold_steps=10`), rather than re-deciding `ru_on/split` every step as in Table A.1’s canonical rows for these same two methods.

Table A.3: Per-seed final evaluation metrics, fair-cadence control: DQN and MP-DQN retrained with BMPP-DQN’s own $N = 10$ discrete hold.

Algorithm ($N=10$)	Seed	Reward	Power (W)	QoS Rate (%)	Per-UE QoS (%)	Switching	Throughput (Mbps)
DQN	42	-104047.70	104.30	19.4	72.90	0.096	39.50
DQN	123	-61684.90	148.00	23.6	77.80	0.084	224.40
DQN	456	-64477.30	123.60	23.2	78.30	0.054	212.50
MP-DQN	42	-64403.50	141.00	23.8	78.40	0.030	220.40
MP-DQN	123	-106382.30	132.40	18.6	72.70	0.086	2.30
MP-DQN	456	-63237.50	201.00	23.6	78.00	0.040	242.30

Every DQN/MP-DQN seed’s switching frequency at $N = 10$ (range 0.030-0.096) is lower than *every* BMPP-DQN canonical seed’s (range 0.124-0.236, Table A.1) – not merely comparable, but consistently *below* it. This is the seed-level evidence behind Section 4.8’s central finding: BMPP-DQN’s switching-frequency advantage over DQN/MP-DQN at their own default (every-step) cadence is substantially a cadence artifact, not an architectural one – simple DQN/MP-DQN, once given BMPP-DQN’s own decision cadence, switch even less than BMPP-DQN itself.